



School of Digital, Technologies and Arts
Department of Computing

COMP60010: Enterprise Cloud and Distributed Web Applications 2

Assignment Specification

Weighted at 50% of the module mark

Pramishka Kannangara

CB012466

Table of Contents

Introduction	8
1.1 Current Monolithic Challenges	9
1.2 Proposed Cloud Native Vision.....	9
2 Cloud Based Web Application Design.....	11
2.1 Purpose of the Proposed Cloud Architecture	11
2.2 Overall Cloud Architecture	11
2.3 Architectural Justification	12
2.4 Microservices Architecture	13
2.5 Containerisation Strategy.....	13
2.6 Serverless Computing Architecture	15
2.7 Cloud Managed Database Architecture	15
2.8 High Availability and Multi Region Deployment	15
2.9 Horizontal Scalability	16
2.10 Real Time Event Driven Integration Architecture	16
3 Distributed Web Application and API Design.....	18
3.1 Distributed System Overview	18
3.2 RESTful API Design	18
3.3 Core Microservices Implementation.....	21
3.4 Payment Service Implementation.....	22
3.5 API Gateway Implementation and Request Routing	26
3.6 API Documentation (OpenAPI / Swagger)	27
3.7 Event-Driven Architecture using AWS EventBridge	30
3.7.1 Event Publication and Routing	30
3.7.2 Event Processing and Notification Generation.....	31
3.8 Secure Service to Service Communication	33
3.8.1 JWT Based Authentication and Authorization.....	33
3.8.2 HTTPS and API Gateway Security Controls	34
3.9 Frontend User Interface Implementation	34
4 Data Security, Compliance and Consistency	39

4.1 Encryption at Rest.....	39
4.2 Encryption in Transit.....	39
4.3 Authentication and Authorization	40
4.4 GDPR Compliance Considerations	40
4.5 PCI-DSS Compliance Considerations	40
4.6 Data Consistency Strategy	40
4.7 Saga Pattern Justification.....	41
5 Real-Time Data Synchronisation.....	43
5.1 Flight Seat Availability Updates	43
5.2 Baggage Status Synchronization	43
5.3 Flight Schedule Updates.....	44
5.4 Pricing Updates.....	44
5.5 Synchronization Results	45
6 Fault Tolerance and Resilience.....	46
6.1 Retry Policies.....	46
6.2 Dead Letter Queues.....	46
6.3 Circuit Breaker Pattern	47
6.4 Load Balancing	48
6.5 Auto Scaling	48
6.6 Multi-AZ Deployment.....	49
6.7 Disaster Recovery	49
7 Performance and Scalability Testing	50
7.1 Testing Environment.....	50
7.2 Load Testing Methodology.....	50
7.3 Load Testing Results.....	50
7.4 Stress Testing Results.....	52
7.5 Throughput Analysis	53
7.6 Error Rate Analysis	53
7.7 Performance Improvements	54
8 Monitoring and Observability	55
8.1 AWS CloudWatch Monitoring.....	55

8.2 Application Logging	55
8.3 Metrics Collection	56
8.4 Distributed Tracing	57
8.5 Fault Diagnosis Example	58
9 Testing Strategy	60
9.1 Unit Testing	60
9.2 Integration Testing	61
9.3 API Testing.....	62
9.4 Security Testing	66
9.5 End-to-End Testing	67
10 Challenges Encountered.....	68
11 Future Improvements	69
12 Conclusion	70
Appendix - System Access Links.....	71
References	72

Table of Figures

Figure 1: Overall AeroLink Cloud Architecture	11
Figure 2: Docker container images created and tagged for deployment to Amazon ECR.	14
Figure 3:Amazon ECS Fargate cluster running AeroLink containerised microservices.	14
Figure 4:Amazon Elastic Container Registry repositories storing AeroLink container images. .	14
Figure 5:Amazon DynamoDB tables used by AeroLink microservices for distributed data storage and persistence.....	15
Figure 6:ECS Fargate tasks deployed across multiple Availability Zones	16
Figure 7:Amazon SQS queue configured to support asynchronous message processing	17
Figure 8:ECS Fargate Cluster Hosting AeroLink Microservices.....	18
Figure 9:API Gateway Request Routing Process.....	19
Figure 10:API Gateway request routing workflow.....	20
Figure 11:API Gateway Route Configuration	20
Figure 12:API Gateway request routing workflow.....	21
Figure 13:Amazon ECS Fargate cluster overview	22
Figure 14:AWS Lambda functions implementing AeroLink microservices.	22
Figure 15:AWS Lambda Payment Service Function	23
Figure 16:Payment Service API Gateway Route Configuration	24
Figure 17:Payment Service Integration with AWS Lambda.....	24
Figure 18:Successful Payment Authorization Response.....	25
Figure 19:Payment Transactions Stored in DynamoDB.....	25
Figure 20:API Gateway Integrations with ECS and AWS Lambda Services.....	26
Figure 21:Production API Gateway Deployment Endpoint	27
Figure 22:Swagger UI documentation for AeroLink REST APIs.....	28
Figure 23:Swagger API execution example demonstrating endpoint testing (flights).	29
Figure 24:Swagger API execution example demonstrating endpoint testing (checkins).	29
Figure 25:AeroLink project structure containing OpenAPI documentation	30
Figure 26: Open API yaml code	30
Figure 27:EventBridge rule configured to route baggage events to the notification service.	31
Figure 28:Baggage service generating events that are published to Amazon EventBridge.....	31
Figure 29:Notification records automatically generated after EventBridge event processing. ...	32
Figure 30:CloudWatch logs showing successful processing of EventBridge events by the notification service.	32
Figure 31:Successful JWT Token Generation	33
Figure 32:Protected API Documentation	33
Figure 33:Amazon API Gateway provides secure HTTPS endpoints for communication between AeroLink microservices.....	34

Figure 34:User Login Interface	35
Figure 35:User Register Interface	35
Figure 36:Dashboard Overview	36
Figure 37:Flight Management Interface	36
Figure 38:Passenger Booking and Payment Interface	37
Figure 39:Passenger Check in Interface and baggage tracking	37
Figure 40:Airport Integration, Event Management Interface and User management	38
Figure 41:Notification section	38
Figure 42:DynamoDB encryption at rest using AWS-owned encryption keys.....	39
Figure 43:Secure HTTPS endpoint provided by Amazon API Gateway using TLS encryption.	39
Figure 44:JWT token issued after successful user authentication.	40
Figure 45:Secure payment authorization process supporting PCI-DSS compliance requirements.	40
Figure 46:Eventual Consistency Model using EventBridge	41
Figure 47:Saga Transaction Flow	42
Figure 48:Booking creation screen showing selected flight and number of seats.	43
Figure 49:Flight record after booking showing reduced available seats.	43
Figure 50:EventBridge rule routing baggage update events to the Notification Service.	44
Figure 51:DynamoDB notification record generated after baggage status update.	44
Figure 52:Updated flight record in DynamoDB.....	44
Figure 53:DynamoDB flight table showing updated price.	45
Figure 54:Frontend retry logic used for temporary API failures.	46
Figure 55:EventBridge rule used for event delivery retry handling.	46
Figure 56:Amazon SQS Dead Letter Queue configuration for failed messages.	47
Figure 57:Circuit breaker logic used to stop repeated calls to a temporarily failing service.	47
Figure 58:Amazon CloudFront distribution improving availability and traffic distribution.....	48
Figure 59:ECS Fargate services supporting independent scaling of AeroLink microservices.	48
Figure 60:DynamoDB on demand capacity supporting automatic database scaling.....	48
Figure 61:ECS services deployment across multiple Availability Zones to improve availability.	49
Figure 62:Proposed Disaster Recovery Strategy.....	49
Figure 63:k6 load testing script	50
Figure 64:load-test.js result screenshot	51
Figure 65:booking-test.js result screenshot.....	51
Figure 66:k6 Stress testing script.....	52
Figure 67:stress-test.js result screenshot	53
Figure 68:CloudWatch dashboard displaying Lambda performance metrics.	55
Figure 69:CloudWatch log groups for AeroLink microservices.....	56

Figure 70:Lambda execution logs captured in CloudWatch.....	56
Figure 71:CloudWatch metrics collected during performance testing.	57
Figure 72:AWS X-Ray service map showing distributed tracing across AeroLink cloud services.	58
Figure 73:AWS Lambda Active Tracing enabled for distributed tracing using AWS X-Ray.....	58
Figure 74:Fault diagnosis workflow using CloudWatch logs and traces.....	59
Figure 75:Notification records generated after baggage status updates.	59
Figure 76:Automated Jest unit testing results for AeroLink microservices	61
Figure 77:Payment transaction records successfully stored in Amazon DynamoDB following integration testing of the AeroLink Payment Service.....	62
Figure 78:Baggage Service API Testing	63
Figure 79:Check-In Service API Testing	64
Figure 80:Notification Service API Testing.....	65
Figure 81:Automated API test execution results using Axios for AeroLink microservices.	66
Figure 82:JWT token generation (/auth/login)	66
Figure 83:Protected endpoint access with valid JWT (HTTP 200).....	66
Figure 84:Unauthorized access attempt (Access denied. Valid JWT token required.).....	66

List of Tables

1:Real Time Synchronisation Scenarios and Expected Outcomes	45
2:Load Testing Results for AeroLink APIs	52
3:Stress Testing Results and System Limits	53
4:Summary of Testing Activities and Outcomes	60
5:Integration Testing Summary	61
6:API Testing Results for AeroLink Microservices.....	65

Introduction

1.1 Current Monolithic Challenges

The AeroLink platform is currently using a traditional monolithic architecture that incorporates all business domains, such as flight booking, passenger boarding, luggage handling, and flight management, in a single deployment package. Although this design approach may have been adequate for the first stage of development, today, it is no longer able to cope with the strict requirements associated with being a technology provider for the airline industry on a global scale. Due to the fact that all application components operate within the same computing boundary and database, the current system finds it extremely difficult to ensure high availability and fault tolerance. Any runtime error, memory corruption issue, or failure during the integration with third-party applications, such as airports, border control facilities, or banking organizations, will propagate through the entire application boundary, causing critical failures and system crashes throughout the whole process.

Moreover, the one-dimensional design imposes very high thresholds for scalability and optimization. In order to cope with a large influx of users from different regions during the time of peak travel or a sudden spike in bookings during particular seasons, the company has to perform scalability on the entire system stack of applications regardless of the affected micro-domains. As a result, AeroLink suffers from major architectural inefficiencies.

As mentioned above, scalability limitations of the application are further compounded by the inability of the platform to support real-time data synchronization. In an international air travel system, essential changes such as availability of seats in flights, status of luggage, and modifications in the schedule or price must reach all the international airports with minimal time delay. Doing any kind of modifications on data through the monolithic approach results in serious problems related to data locking, race conditions, and data synchronization. Therefore, there is a need to move from a monolithic approach to a cloud-native one.

1.2 Proposed Cloud Native Vision

To overcome the significant restrictions that characterize the existing monolithic structure, it is recommended to switch to the cloud-native design with a focus on microservices architecture that provides the integration of distributed applications and cloud services through scalable and secure APIs. The proposed solution will be based on breaking down the monolithic application into a set of loosely coupled and independent microservices aligned with specific business capabilities, namely, flights reservation, boarding pass, bag tracking, and flights operations management. With the help of containerization using such technologies as Docker for running core workloads continuously alongside serverless architectures represented by AWS Lambda for executing events, each domain will be scalable and manageable independently. At the same time, an API Gateway at the edge of the infrastructure will provide for request routing, rate limiting, and edge security.

Moreover, this migration to a cloud native solution also upgrades the data layer by adopting a move from a centralized shared database into a decentralized and cloud-managed databases, which are specifically tailored to individual microservices' requirements. Transactional relational databases like Amazon Aurora would be employed to carry out transactions with ACID semantics, such as ticketing and bookings, while high performance low latency NoSQL databases like Amazon DynamoDB would handle transactional information. In order to address the issue of real-time data synchronization in the organization, the proposed architecture includes a message broker through an event-driven design using technology like AWS EventBridge (GeeksforGeeks, 2024). This message backbone allows the streaming of real-time events such as seat availability in flights, real-time monitoring of luggage, and price changes with negligible delay. By being deployed in a multi-regional fashion, this architecture removes all single points of failures and ensures high availability.

2 Cloud Based Web Application Design

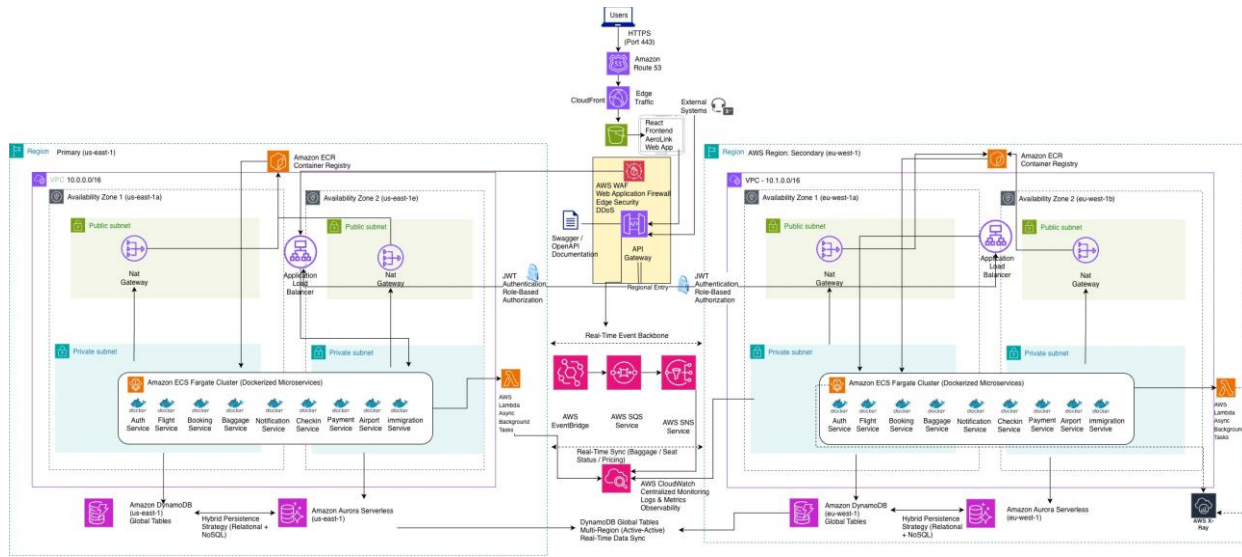


Figure 1: Overall AeroLink Cloud Architecture

2.1 Purpose of the Proposed Cloud Architecture

The AeroLink proposed cloud-based system design is a robust, ready-for-production airline systems environment that will cater for airline activities all around the globe, and will have high transaction volumes and real-time updates, among other qualities. With AeroLink having vital functions such as booking flights, luggage tracking, managing flights and passengers, it needs to be a highly available, scalable, secure, and fault-tolerant system.

The new architecture transforms AeroLink's current architecture into one which is cloud-native and distributed using Amazon Web Services. The system features an architecture that comprises microservices, serverless, database services, edge computing, API management, monitoring, and disaster recovery across multiple regions.

2.2 Overall Cloud Architecture

The production architecture follows a multi-layered cloud approach. The first level is that of the edge, where the AeroLink web application can be accessed by users via HTTPS with port 443. The DNS routing service is provided by Amazon Route 53, whereas the distribution of the application is performed using Amazon CloudFront. The AWS WAF layer shields the system from web attacks and malicious threats (K21 Academy, 2023).

Frontend application is provided by the static React web application running on Amazon S3 using CloudFront service. The frontend application requests to the APIs are provided through Amazon API Gateway. Amazon API Gateway provides access to backend services through routing, authorization, throttling, and providing access to API services.

The proposed API Gateway Architecture design employs Amazon ECS Fargate to deploy the containers as microservices. Routing of traffic takes place from the Application Load Balancer to various Availability Zone-based deployment of microservices. The async calls can take place through AWS Lambda, EventBridge, SQS, and SNS. The data storage involves the use of Amazon DynamoDB and Amazon Aurora Serverless, based on the consistency and transaction requirements of each domain.

2.3 Architectural Justification

The AeroLink platform uses a kind of design that is made for the cloud. This design is built using Amazon Web Services that were picked because they can handle a lot of users work well and are reliable. For the parts of the platform Amazon ECS Fargate was used because it takes care of the servers and gives more control over the work that the platform does (Amazon.com, 2019). This is better than using fashioned virtual machines.

Amazon ECS Fargate and other services like AWS Lambda were chosen because they can handle a lot of work without needing a lot of management. AWS Lambda is also good for jobs that need to be done when something happens because it only uses resources when it is actually doing something.

The AeroLink platform uses Amazon API Gateway as the door for all the application programming interfaces. This is because it can handle a lot of things like making sure the right requests get to the place stopping too many requests checking who is making the requests and keeping an eye on everything.

The platform uses Amazon DynamoDB to store a lot of data that it needs every day. This is because Amazon DynamoDB is very fast and can handle a lot of users. For data that needs to be treated like a database the platform uses Amazon Aurora Serverless. Amazon Aurora Serverless is good, at handling transactions and keeping Amazon DynamoDB and other data safe (Amazon.com, 2026). The platform picked Amazon DynamoDB and Amazon Aurora Serverless for these reasons

The AeroLink platform uses Amazon EventBridge to help all the parts of the platform talk to each other. This is better than having each part talk to each other because it makes the platform more flexible and easier to manage. The platform also uses Amazon CloudFront, Route 53 and AWS WAF to make sure that users (Amazon Web Services, Inc., 2026), around the world can get to the platform quickly and safely.

All of these services work together to make a platform that's strong can handle a lot of users and does not need a lot of work to keep it running. This is what the AeroLink platform needs to work for an airline.

2.4 Microservices Architecture

Architecture used in AeroLink consists of cloud-native microservices architecture where each microservice performs the business capability on its own. Cloud-native microservices provide maintainability, scalability, and ease of deployment in comparison with monolithic architecture.

Authentication Service deals with user sign-up, authentication of users, JWT tokens creation, OAuth 2.0 management, and RBAC management. It is an essential part of the platform as far as it provides identification of regular and administrative users.

Flight Services take care of flight scheduling, flight pricing, availability of seats, routing and flight statuses. It facilitates operations in an airline and provides passengers and employees with flight details at any time.

Booking Services provide ticketing, seat assignment, booking confirmation, and passenger reservation services. The Booking Service is connected to the Flight Services for verification of seat availability before booking.

Baggage Service is responsible for performing operations such as baggage registration, baggage tracking, and status notification related to baggage.

Notification Service deals with events that take place during the processing and dissemination of notifications to users. Such notifications may consist of booking confirmation, baggage status update, flight delay, gate change, etc.

In addition to the airline microservices discussed above, there are other microservices used by the AeroLink system to give extra functionality. One of them is Check-In Microservice, which is charged with the task of carrying out check-ins of passengers and providing them with their boarding passes, whereas the other is Payment Microservice that processes payments made by the customers through integration with third-party payment services.

Furthermore, the software offers Airport Integration Services and Immigration Integration Services to assist in communications between the system and other external organizations. The Airport Integration Services deal with gate assignments, boarding of flights, and airport operations communications while Immigration Integration Services deal with the verification of passenger travel status and immigration status prior to boarding. This shows that AeroLink has the capability to integrate with third parties in order to meet its operational requirements.

2.5 Containerisation Strategy

In AeroLink, a containerisation approach has been adopted to encapsulate microservices along with their respective dependencies and runtime environment through Docker. The approach provides consistency among the different stages in the application's lifecycle, i.e., development, testing, and production. In addition to improving efficiency, it prevents any kind of compatibility problems during the deployment of microservices in the production environment.

To make use of the design suggested above for the deployment of the microservices in the system called AeroLink, the Amazon ECS Fargate services have to be applied. Container images for the microservices will be hosted in the Amazon Elastic Container Registry. As there is no need for managing the Virtual Machine due to the characteristics of the ECS Fargate service, the ECS Fargate service can ensure other properties as well.

```
pramishkakkannagaram2@192 frontend % docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE
084051596652.dkr.ecr.us-east-1.amazonaws.com/auth-service:latest	2992eb2d2200	50.9MB	50.9MB
084051596652.dkr.ecr.us-east-1.amazonaws.com/baggage-service:latest	4e03d859937f	53.1MB	53.1MB
084051596652.dkr.ecr.us-east-1.amazonaws.com/booking-service:latest	7a0994f6db49	53.1MB	53.1MB
084051596652.dkr.ecr.us-east-1.amazonaws.com/flight-service:latest	7def990bb9db	50.5MB	50.5MB
084051596652.dkr.ecr.us-east-1.amazonaws.com/notification-service:latest	727cbdbc8873	50.5MB	50.5MB

Figure 2: Docker container images created and tagged for deployment to Amazon ECR.

The screenshot shows the Amazon ECS console for the 'aerolink-cluster'. The cluster is active and has 5 running tasks. Below the overview, a table lists the services:

Service name	ARN	Status	Scheduling strategy	Launch type	Task definition	Deployments and tasks	Last deployment	Created at
auth-service	arn:aws:ecs:us-east-1:084051596652:cluster/aerolink-cluster/task-definition/auth-service-task:1	Active	Replica	Fargate	auth-service-task:1	1/1 Tasks running	Completed	7 days ago
baggage-service	arn:aws:ecs:us-east-1:084051596652:cluster/aerolink-cluster/task-definition/baggage-service-task:1	Active	Replica	Fargate	baggage-service-task:1	1/1 Tasks running	Completed	7 days ago
booking-service	arn:aws:ecs:us-east-1:084051596652:cluster/aerolink-cluster/task-definition/booking-service-task:1	Active	Replica	Fargate	booking-service-task:1	1/1 Tasks running	Completed	7 days ago
flight-service	arn:aws:ecs:us-east-1:084051596652:cluster/aerolink-cluster/task-definition/flight-service-task:1	Active	Replica	Fargate	flight-service-task:1	1/1 Tasks running	Completed	7 days ago
notification-service	arn:aws:ecs:us-east-1:084051596652:cluster/aerolink-cluster/task-definition/notification-service-task:1	Active	Replica	Fargate	notification-service-task:1	1/1 Tasks running	Completed	7 days ago

Figure 3: Amazon ECS Fargate cluster running AeroLink containerised microservices.

The screenshot shows the Amazon ECR console for the 'Private repositories' section. A table lists the repositories:

Repository name	URI	Created at	Tag immutability	Encryption type
auth-service	084051596652.dkr.ecr.us-east-1.amazonaws.com/auth-service	May 28, 2026, 10:05:28 (UTC+05:30)	Mutable	AES-256
baggage-service	084051596652.dkr.ecr.us-east-1.amazonaws.com/baggage-service	May 28, 2026, 10:05:34 (UTC+05:30)	Mutable	AES-256
booking-service	084051596652.dkr.ecr.us-east-1.amazonaws.com/booking-service	May 28, 2026, 10:05:32 (UTC+05:30)	Mutable	AES-256
flight-service	084051596652.dkr.ecr.us-east-1.amazonaws.com/flight-service	May 28, 2026, 10:05:30 (UTC+05:30)	Mutable	AES-256
notification-service	084051596652.dkr.ecr.us-east-1.amazonaws.com/notification-service	May 28, 2026, 10:05:36 (UTC+05:30)	Mutable	AES-256

Figure 4: Amazon Elastic Container Registry repositories storing AeroLink container images.

2.6 Serverless Computing Architecture

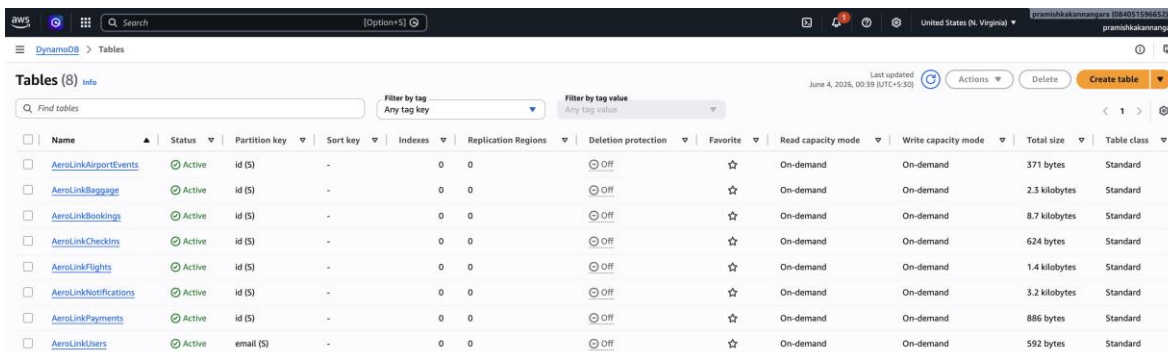
Other than the continual operation of microservices, serverless computing forms a part of the AeroLink architecture through AWS Lambda. Serverless computes provide event-driven operations, batch processes, automation processes, and integrations that need no special hardware infrastructure at all.

The invocation of AWS Lambda function may be done through Amazon API Gateway, Amazon EventBridge, Amazon Simple Notification Service (Amazon SNS), and Amazon Simple Queue Service (Amazon SQS). This may involve notification processes, asynchronous validations, event processes, and system integrations.

2.7 Cloud Managed Database Architecture

Architecture for the AeroLink project is characterized by mixed persistence which involves the use of Amazon Aurora Serverless together with Amazon DynamoDB. As such, each specific domain can utilize the best technology depending on the requirements.

In particular, transactions that need reliability and consistency like booking for flights or tickets and information regarding passengers will be done using Amazon Aurora Serverless technology. On the other hand, operational data like baggage management, notifications for future events taking place in airports will be done using Amazon DynamoDB technology that enables replication of data globally with the aid of Global Tables.



Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode	Total size	Table class
AeroLinkAirportEvents	Active	id (S)	-	0 0	0 0	Off	☆	On-demand	On-demand	371 bytes	Standard
AeroLinkBaggage	Active	id (S)	-	0 0	0 0	Off	☆	On-demand	On-demand	2.3 kilobytes	Standard
AeroLinkBookings	Active	id (S)	-	0 0	0 0	Off	☆	On-demand	On-demand	8.7 kilobytes	Standard
AeroLinkCheckins	Active	id (S)	-	0 0	0 0	Off	☆	On-demand	On-demand	624 bytes	Standard
AeroLinkFlights	Active	id (S)	-	0 0	0 0	Off	☆	On-demand	On-demand	1.4 kilobytes	Standard
AeroLinkNotifications	Active	id (S)	-	0 0	0 0	Off	☆	On-demand	On-demand	3.2 kilobytes	Standard
AeroLinkPayments	Active	id (S)	-	0 0	0 0	Off	☆	On-demand	On-demand	886 bytes	Standard
AeroLinkUsers	Active	email (S)	-	0 0	0 0	Off	☆	On-demand	On-demand	592 bytes	Standard

Figure 5: Amazon DynamoDB tables used by AeroLink microservices for distributed data storage and persistence.

2.8 High Availability and Multi Region Deployment

The high availability of the software is obtained via its deployment in two AWS regions, such as US East (N. Virginia – us-east-1), which serves as a production region and Europe (Ireland – eu-west-1) that operates as a disaster recovery region. In turn, multiple availability zones have been implemented in order to avoid a potential single point of failure. As a result, ECS Fargate microservices as well as Application Load Balancers are deployed across different zones to provide uninterrupted operation of services even in case some infrastructure elements break down.

DNS-based failover is provided by using Amazon Route 53 to switch traffic from one region to another in the event of an outage. The software provides application content globally with low latency using CloudFront and ensures synchronization of transactional data across both regions using DynamoDB Global Tables and Aurora Serverless replication.

Tasks (5)

Filter tasks by property or value

Filter desired status: Any desired status

Filter launch type: Any launch type

Task definition	Health status	Started by	Created at	Started at	Group	Container instances	Launch type	Platform ...	CPU	Memory	Availability zone
notification-service-task:1	Unknown	ecs-svc/26735625904...	7 days ago	7 days ago	service:notification-service	-	Fargate	1.4.0	.25 vCPU	.5 GiB	us-east-1e
auth-service-task:1	Unknown	ecs-svc/01541137333...	7 days ago	7 days ago	service:auth-service	-	Fargate	1.4.0	.25 vCPU	.5 GiB	us-east-1e
booking-service-task:1	Unknown	ecs-svc/06758533759...	7 days ago	7 days ago	service:booking-service	-	Fargate	1.4.0	.25 vCPU	.5 GiB	us-east-1e
baggage-service-task:1	Unknown	ecs-svc/71675849950...	7 days ago	7 days ago	service:baggage-service	-	Fargate	1.4.0	.25 vCPU	.5 GiB	us-east-1e
flight-service-task:1	Unknown	ecs-svc/55372467959...	7 days ago	7 days ago	service:flight-service	-	Fargate	1.4.0	.25 vCPU	.5 GiB	us-east-1a

Figure 6: ECS Fargate tasks deployed across multiple Availability Zones

2.9 Horizontal Scalability

This design ensures that horizontal scaling is supported at every layer of the platform architecture. The microservices which are implemented using Amazon ECS Fargate technology are able to scale up or down depending on the workload. This will ensure that the system can be used even when there are many requests during holiday seasons or when flights are canceled.

Yet another characteristic of horizontal scalability is applied in the databases themselves. The scalability in the database layer has been achieved because DynamoDB automatically scales its throughput depending on the workload, while Aurora Serverless scales depending on the workload too. In such a manner, the system can easily scale even with a large number of users accessing it from various places.

2.10 Real Time Event Driven Integration Architecture

The real-time communication between different services is implemented by making use of the event-driven architecture approach, which uses Amazon EventBridge, Amazon SQS, and Amazon SNS. Instead of depending solely on direct service-to-service communication, business events are published and consumed in an asynchronous manner.

Event-driven computing involves operations such as confirming a booking, updating the status of luggage, updating flight schedules, messages from airports, confirming payments, and immigrations. The role of EventBridge comes into play as a backbone for events, SQS is used to ensure queueing, and SNS ensures broadcasting of events to multiple recipients.

AeroLinkDLQ

EditDeletePurgeSend and receive messagesStart DLQ redrive

Detailsinfo

Name

AeroLinkDLQ

Encryption

Amazon SQS key (SSE-SQS)

Type

Standard

URL

https://sqs.us-east-1.amazonaws.com/084051596652/AeroLinkDLQ

ARN

arn:aws:sqs:us-east-1:084051596652:AeroLinkDLQ

Dead-letter queue

-

More

Figure 7: Amazon SQS queue configured to support asynchronous message processing

3 Distributed Web Application and API Design

3.1 Distributed System Overview

The AeroLink application represents a cloud native architecture that is comprised of microservices that can be independently deployed. Each microservice is responsible for carrying out a specific business process and is connected to other microservices using predefined API's. In other words, the architectural approach does not suffer from the limitations of monolithic architecture.

AeroLink's distributed structure includes Authentication Service, Flight Service, Booking Service, Baggage Service, Notification Service, Check-In Service, Payment Service, Airport Service Integration, and Immigration Service Integration. The deployment of microservices is done via Docker Containers in Amazon ECS Fargate and API Gateway that works as the gateway for all client requests.

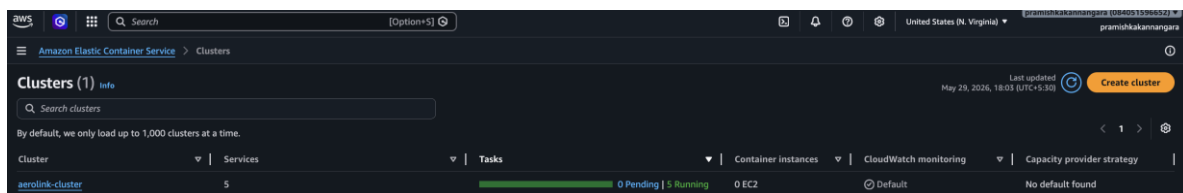


Figure 8:ECS Fargate Cluster Hosting AeroLink Microservices

3.2 RESTful API Design

AeroLink mainly utilizes RESTful APIs in order to achieve communication between the frontend software and microservices in the backend. The microservices expose an array of endpoints that perform various CRUD operations. The frontend software consumes the APIs via Amazon API Gateway to send request to the relevant backend microservice.

REST ensures that there is loose coupling of the services provided as well as interoperability by employing the HTTP methods such as GET, POST, PUT, and DELETE. The protocol used to transfer data in this architecture is JSON, which ensures that the process is platform independent. The consequence of all this is that both passengers and airport employees will be able to interact with the airlines using the same API interface.

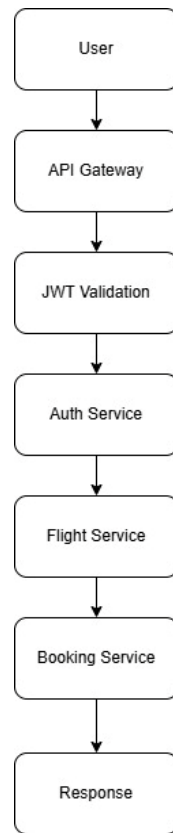


Figure 9:API Gateway Request Routing Process

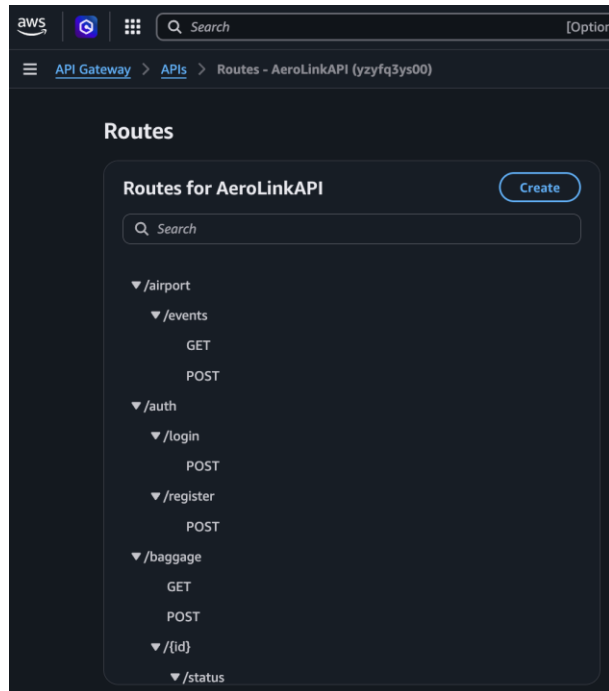


Figure 10:API Gateway request routing workflow

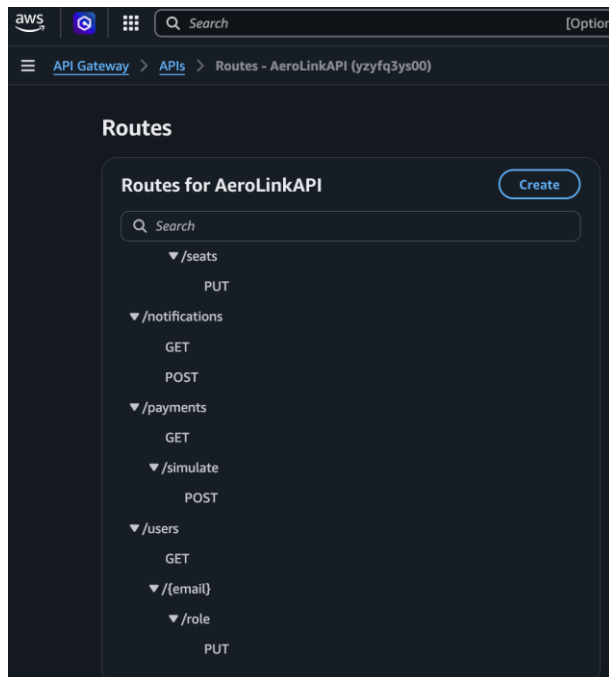


Figure 11:API Gateway Route Configuration

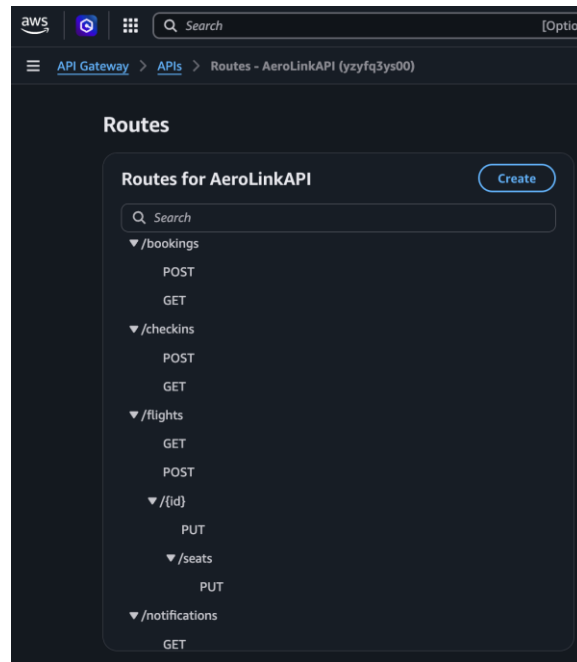


Figure 12:API Gateway request routing workflow

3.3 Core Microservices Implementation

The responsibility of the Authentication Service includes managing user registration, logging in the user, issuing JWT tokens, role validation, and authorization. The Authentication Service can be regarded as the security layer for the entire application since it helps prevent any unauthorized user from accessing any resource within the system. Users with roles such as Passenger, Staff, and Administrator are restricted through RBAC.

The Flight Service handles flight scheduling, flight prices, availability of seats, departure information, arrival information, and status updates for flights. The Administrator and airline staff have the authority to schedule flights whereas the passengers can find out about the flights available before booking.

This service will handle the booking process, ticket issuance, seating arrangements, and confirm the booking made by passengers. Before any kind of confirmation is done, it interfaces with the Flight Service to verify the availability of the needed seats and to calculate the number of available seats.

Baggage Service is responsible for processing the registration of baggage, status updates of baggage, and management of events associated with baggage. If there is any change in the status of the baggage, then events are generated by the Baggage Service automatically via the messaging system.

Notification Service is responsible for processing the operation events and sending notification to the users, which include operation related events, booking confirmation, baggage notifications, flight delay notifications, etc.

AeroLink also provides Payment, Check-In and Airport Integration services. The Payment Service manages transactions, tracks payments and confirms bookings. It's the Check-In Service that takes care of passenger check-ins before departure. Then the Airport Integration Service connects AeroLink to airport systems to share flight information, gate information and to coordinate activities. Each service runs on separate AWS Lambda functions which makes deployment and scaling really easy when needed.

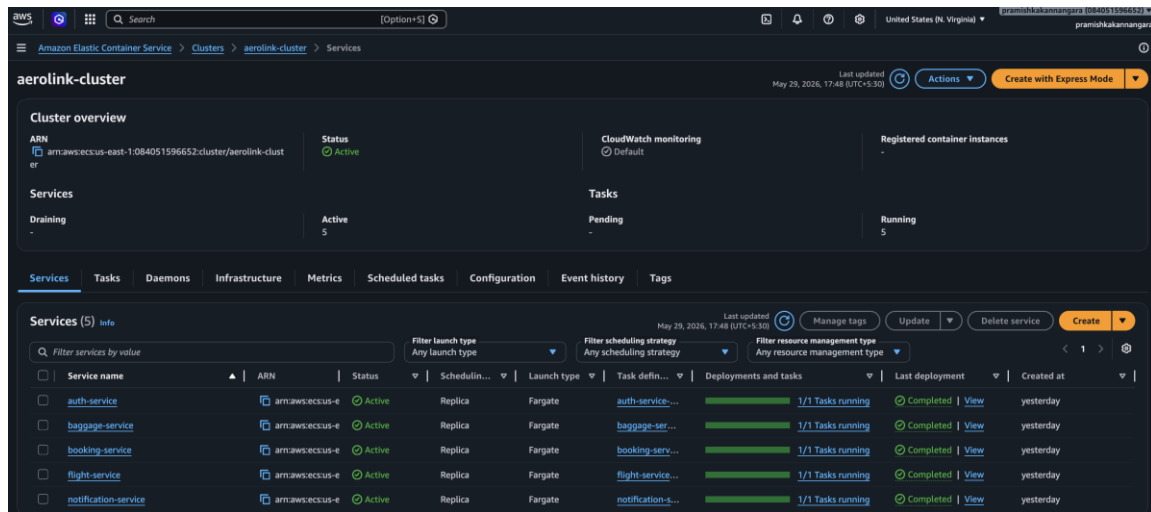


Figure 13: Amazon ECS Fargate cluster overview

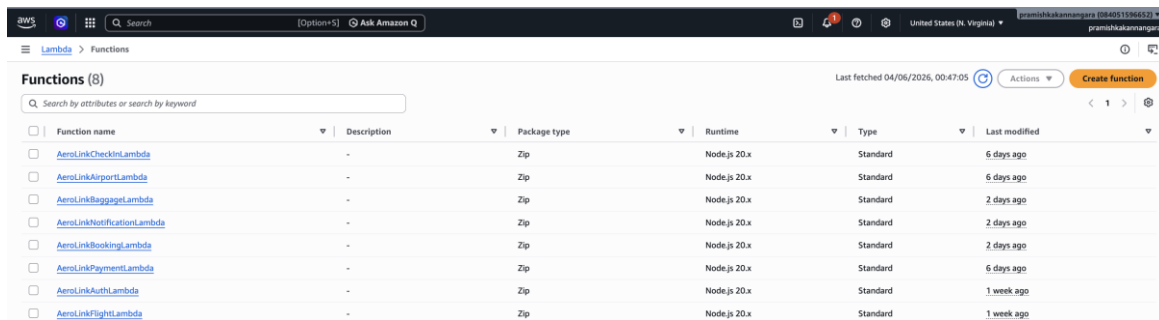


Figure 14: AWS Lambda functions implementing AeroLink microservices.

3.4 Payment Service Implementation

The Payment Service was developed to simulate how the system would interact with third-party payment providers as well as how airline reservation systems would process payments before authorizing the bookings. The service was built on AWS Lambda and integrated with Amazon API Gateway to ensure that the payment requests can be processed through a RESTful API

endpoint. Whenever the passenger reserves their flight ticket, the payment information will be forwarded to the Payment Service which will authorize the payment process through the payment provider.

All the payment details will be saved in Amazon DynamoDB to achieve scalability and high availability for the payment data. Once the payment is authorized, the system will create a unique transaction reference number as well as update the payment status in the workflow. The details regarding the payment will consist of transaction reference number, payment status, provider details, and masked card details.

Integration between the React application frontend and the Payment Service was done through the API Gateway as well. During the process of booking by the passengers, the passengers enter their payment details and create a request for payment. The frontend then sends the request to the Payment Service endpoint, analyzes the response received from the Payment Service, and returns to the passenger the details of confirmation that include transaction ID and approval of payment.

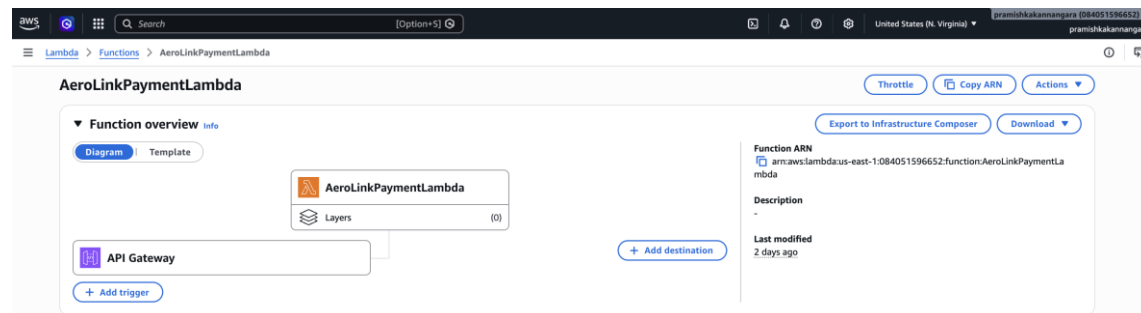


Figure 15: AWS Lambda Payment Service Function

Routes

Routes for AeroLinkAPI

Create

Search

▶ /auth

▶ /baggage

▶ /bookings

▶ /checkins

▶ /flights

▶ /notifications

▼ /payments

GET

▼ /simulate

POST

Figure 16: Payment Service API Gateway Route Configuration

API Gateway > APIs > AeroLinkAPI (c3yfg3yt00) > Integrations

Integrations

Attach integrations to routes Manage integrations

Integrations for AeroLinkAPI

Create

Search

☐ AeroLinkAirportLambda (bat2b0l)

☐ AeroLinkAuthLambda (9baufum)

☐ AeroLinkBaggageLambda (sg7tmea)

☐ AeroLinkBookingLambda (schzpf5)

☐ AeroLinkCheckInLambda (6dnd0c4)

☐ AeroLinkFlightLambda (c3b8lqf)

☐ AeroLinkNotificationLambda (u5cvt6l)

☒ AeroLinkPaymentLambda (wsecpf)

Integration details

Delete Edit

AeroLinkPaymentLambda

Lambda function

AeroLinkPaymentLambda (us-east-1)

Integration ID

wsecpf

Description

-

Payload format version

The parsing algorithm for the payload sent to and returned from your Lambda function. [Learn more](#)

2.0 (interpreted response format)

Invoke permissions

The resource policy of the Lambda function determines if API Gateway can invoke it. You can run the AWS CLI command snippet below to give API Gateway permission to invoke your AWS Lambda function.

Example policy statement

Timeout

The number of milliseconds that API Gateway should wait for a response from the integration before timing out.

30000

Figure 17: Payment Service Integration with AWS Lambda

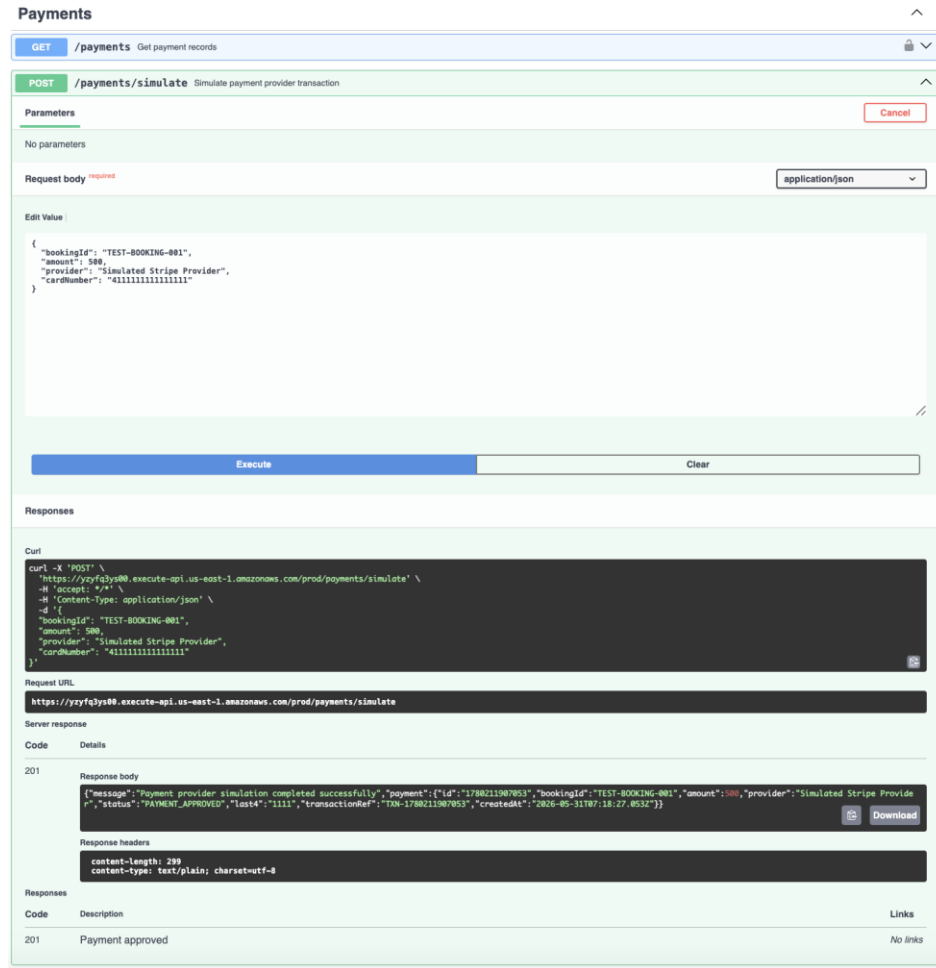


Figure 18: Successful Payment Authorization Response

AeroLinkPayments

Scan or query items

Scan Query

Select a table or index

Table - AeroLinkPayments

Select attribute projection

All attributes

Filters - optional

Run Reset

Table: AeroLinkPayments - Items returned (4)

Scan started on May 31, 2026, 12:52:57

Id (String)	amount	bookingid	createdAt	last4	provider	status	transactionRef
1780033768502	1500	TEMP-17800337...	2026-05-2...	6209	Simulated ...	PAYMENT_...	TXN-1780033768502
1780211907053	500	TEST-BOOKING-...	2026-05-3...	1111	Simulated ...	PAYMENT_...	TXN-1780211907053
1780032788503	500	TEST-BOOKING-...	2026-05-2...	1111	Simulated ...	PAYMENT_...	TXN-1780032788503
1780033838074	500	TEMP-17800338...	2026-05-2...	6209	Simulated ...	PAYMENT_...	TXN-1780033838074

Figure 19: Payment Transactions Stored in DynamoDB

3.5 API Gateway Implementation and Request Routing

Amazon API Gateway was used as the primary interface through which all client requests will be received in the AeroLink distributed system. In other words, users will not interact directly with specific microservices but with the API Gateway which will pass their requests to the corresponding backend services. The solution helps to optimize interactions between users and microservices, making the process more secure and scalable.

Specifically, there is a route in the API Gateway responsible for the work of Authentication, Flight Management, Booking Management, Baggage Tracking, Notifications, Payments, Passenger Check-In, Airport Integration, and Immigration Integration microservices. Requests made by clients are validated based on JWT authentication and then are redirected to the necessary backend service or AWS Lambda function.

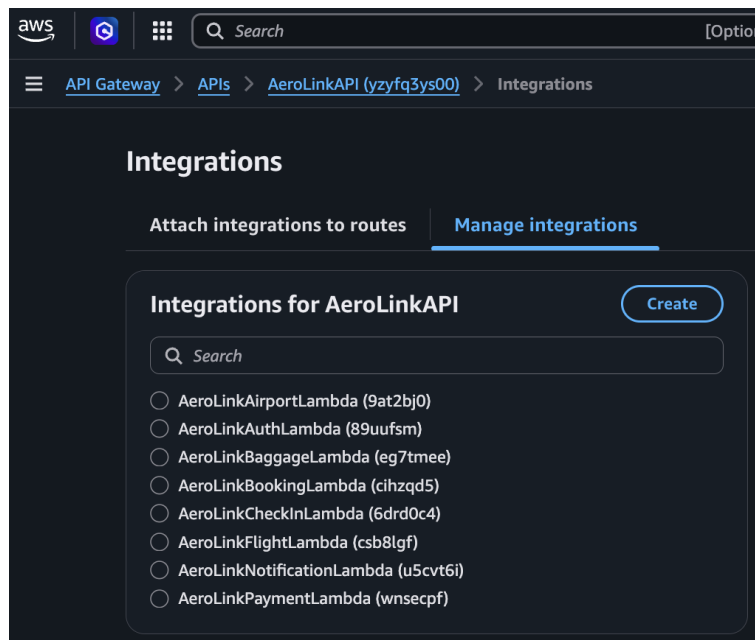


Figure 20:API Gateway Integrations with ECS and AWS Lambda Services

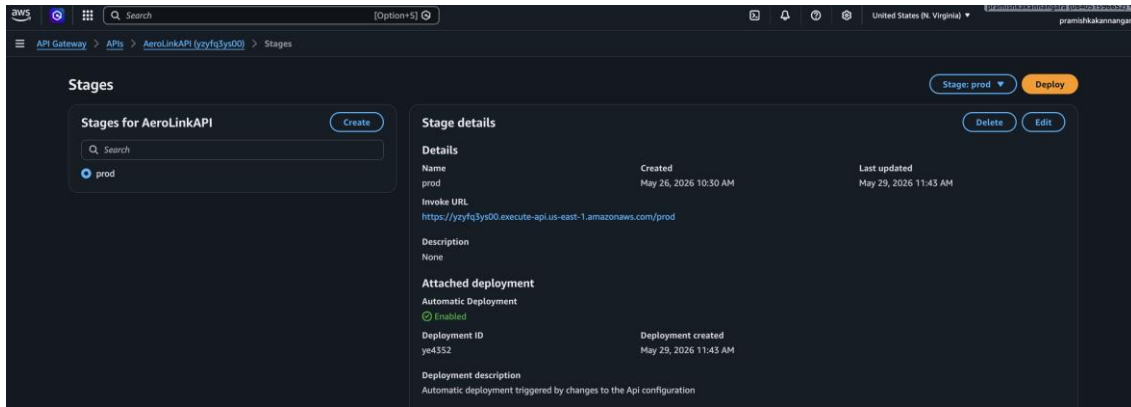


Figure 21: Production API Gateway Deployment Endpoint

3.6 API Documentation (OpenAPI / Swagger)

In order to improve maintainability and make sure that the integration with the system becomes easy, AeroLink makes use of OpenAPI (Swagger) documentation. This API documentation will be helpful since it gives the user information about the various endpoints along with their request methods, parameters accepted, authentication, and response. This document will also help other developers since the document will make integration with AeroLink applications possible even without access to the source code.

Swagger documentation has been created for several AeroLink services including those involved in Authentication, Flight, Booking, Baggage, Notifications, Payments, Passenger Check-In, Airport Integration, and Immigration Integration. Every API endpoint comes along with details on the allowed operations, request payloads, expected response, and status codes.

AeroLink Airline Systems API v1.0 OAuth 2.0

[Swagger UI](#)

Cloud-based distributed microservices API for AeroLink airline operations.

Servers

<https://xyz123y00.execute-api.us-east-1.amazonaws.com/prod> - AWS API Gateway production endpoint

Authorize

Auth

POST	/auth/register	Register a new user	⌵
POST	/auth/login	Login user and return JWT token	⌵

Flights

GET	/flights	Get all flights	⌵
POST	/flights	Create flight	🔒
PUT	/flights/{id}	Update flight schedule, price, or status	🔒

Bookings

GET	/bookings	Get all bookings	🔒
POST	/bookings	Create a booking	🔒

Baggage

GET	/baggage	Get baggage records	🔒
POST	/baggage	Create baggage record	🔒
PUT	/baggage/{id}/status	Update baggage status	🔒

Notifications

GET	/notifications	Get notifications	🔒
POST	/notifications	Create notification	🔒

Payments

GET	/payments	Get payment records	🔒
POST	/payments/simulate	Simulate payment provider transaction	⌵

Check-in

GET	/checkins	Get passenger check-ins	🔒
POST	/checkins	Passenger check-in and boarding pass generation	🔒

Airport Integration

GET	/airport/events	Retrieve airport operational events	🔒
POST	/airport/events	Create airport operational event	🔒

Immigration Integration

GET	/immigration/verify	Retrieve immigration verification records	🔒
POST	/immigration/verify	Verify passenger immigration clearance	🔒

Figure 22: Swagger UI documentation for AeroLink REST APIs.

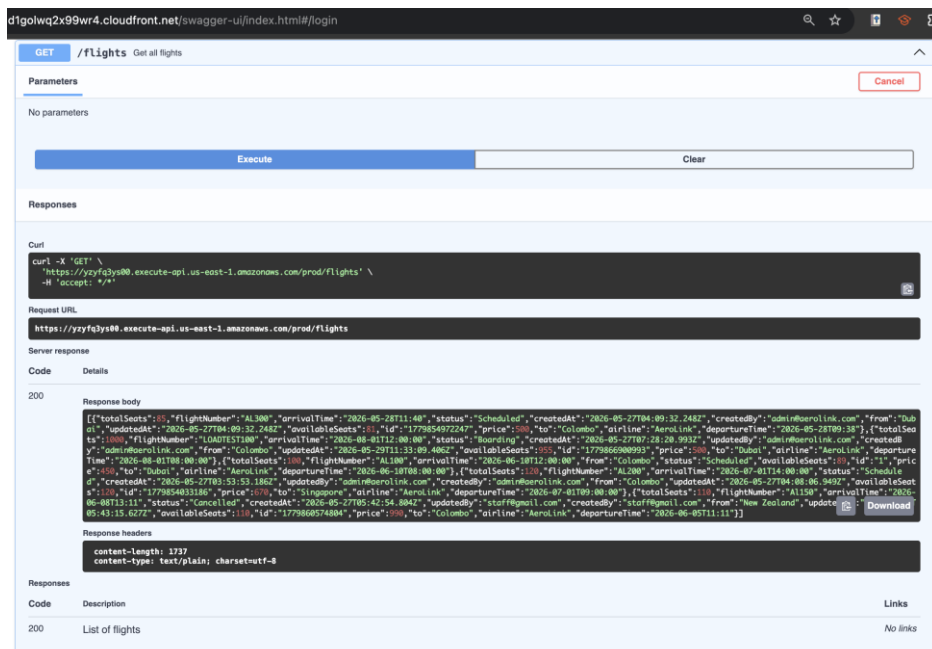


Figure 23:Swagger API execution example demonstrating endpoint testing (flights).

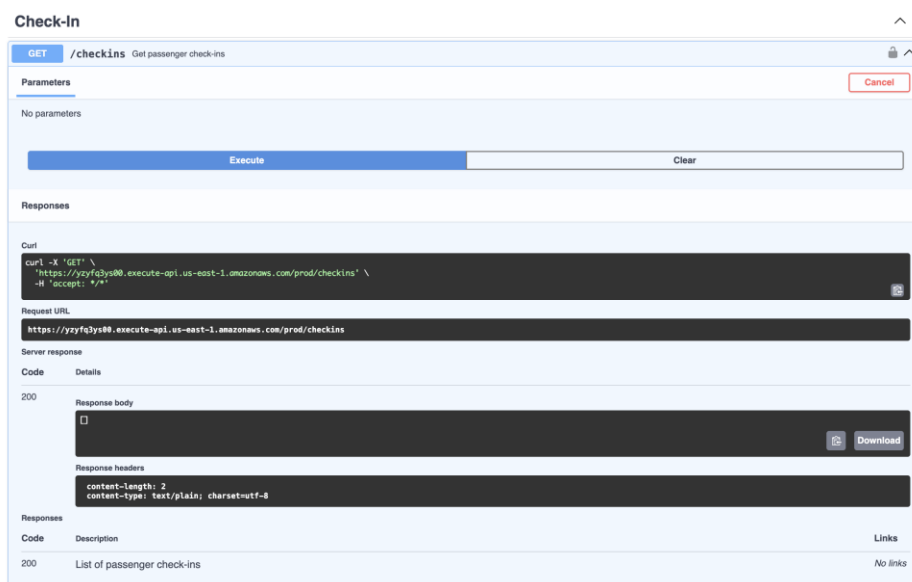


Figure 24:Swagger API execution example demonstrating endpoint testing (checkins).

Index of aerolink-platform/

api-gateway/	auth-task.json	aws-lambda/	baggage-task.json
booking-task.json	docker-compose.yml	ecs-task-trust-policy.json	flight-task.json
frontend/	node_modules/	notification-task.json	openapi/
package-lock.json	package.json	performance-tests/	services/
swagger-ui/	testing-evidence/	tests/	

Figure 25: AeroLink project structure containing OpenAPI documentation

```
openapi > aerolink-api.yaml
1  openapi: 3.0.0
2
3  info:
4    title: AeroLink Airline Systems API
5    version: 1.0.0
6    description: Cloud-based distributed microservices API for AeroLink airline operations.
7
8  servers:
9    - url: https://zyfq3ys00.execute-api.us-east-1.amazonaws.com/prod
10      description: AWS API Gateway production endpoint
11
12  security:
13    - bearerAuth: []
14
15  paths:
16    /auth/register:
17      post:
18        summary: Register a new user
19        tags: [Auth]
20        security: []
21        requestBody:
22          required: true
23          content:
24            application/json:
25              example:
26                name: Admin User
27                email: admin@aerolink.com
28                password: Admin123
29                role: admin
30        responses:
31          "201":
32            description: User registered successfully
```

Figure 26: Open API yaml code

3.7 Event-Driven Architecture using AWS EventBridge

3.7.1 Event Publication and Routing

The use of event-driven architecture that employs Amazon EventBridge facilitates the loose coupling required for the microservices in the AeroLink application. The architecture avoids direct communication through REST calls; instead, it involves publishing business events to an event bus in Amazon EventBridge.

In the design of the solution that will be used in this proposal, the Baggage service is set to emit a new event called `BAGGAGE_UPDATED` whenever there is any update concerning the status of the baggage. The event triggers further processes since Amazon EventBridge sends the event to other services according to specific event rules. The receiving service acts independently and does not have to understand how the Baggage service works.

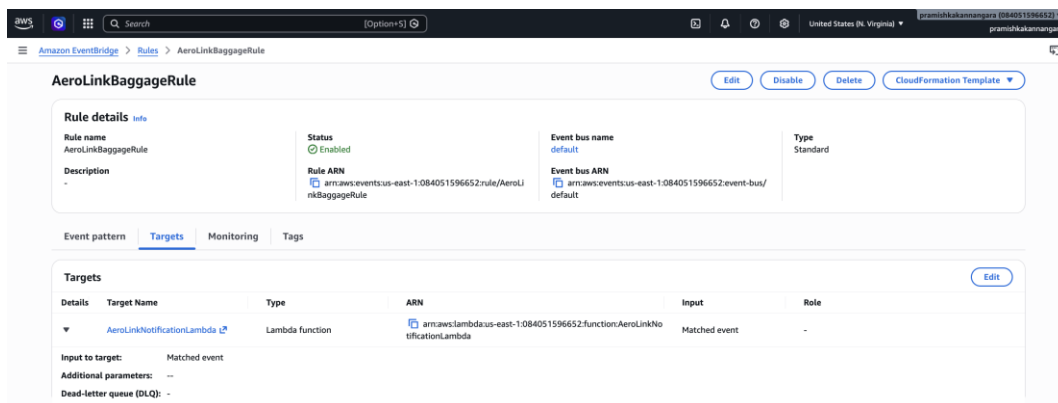


Figure 27: EventBridge rule configured to route baggage events to the notification service.

```
pramishkakannagaram2@192 baggage-lambda % aws events put-rule \
--name AeroLinkBaggageRule \
--event-pattern '{
  "source": ["aerolink.baggage"],
  "detail-type": ["BAGGAGE_UPDATED"]
}' \
--region us-east-1
{
  "RuleArn": "arn:aws:events:us-east-1:084051596652:rule/AeroLinkBaggageRule"
}
pramishkakannagaram2@192 baggage-lambda % aws events put-targets \
--rule AeroLinkBaggageRule \
--targets "Id"="1","Arn"="arn:aws:lambda:us-east-1:084051596652:function:AeroLinkNotificationLambda" \
--region us-east-1
{
  "FailedEntryCount": 0,
  "FailedEntries": []
}
pramishkakannagaram2@192 baggage-lambda % curl -X POST "https://yzfyq3ys00.execute-api.us-east-1.amazonaws.com/prod/baggage" \
-H "Content-Type: application/json" \
-d '{
  "bookingId": "1",
  "passengerName": "Pramishka",
  "tagNumber": "BAG-AL100-001"
}'
{"message": "Baggage record created successfully", "baggage": {"id": "1779790578157", "bookingId": "1", "passengerName": "Pramishka", "tagNumber": "BAG-AL100-001", "status": "Checked-In", "lastUpdated": "2026-05-26T10:16:18.157Z"}}
```

Figure 28: Baggage service generating events that are published to Amazon EventBridge.

3.7.2 Event Processing and Notification Generation

After the receipt of an event in EventBridge, it is subsequently delivered to the Notification service for subsequent processing. Here, the notification microservice accepts the event as input and creates a notification entry in DynamoDB. Asynchronous processing implies the use of events in communicating services rather than the use of API calls.

The CloudWatch logs clearly show that the event was received successfully and processed. In addition, DynamoDB logs further indicate that notifications were created automatically upon updates on baggage status.

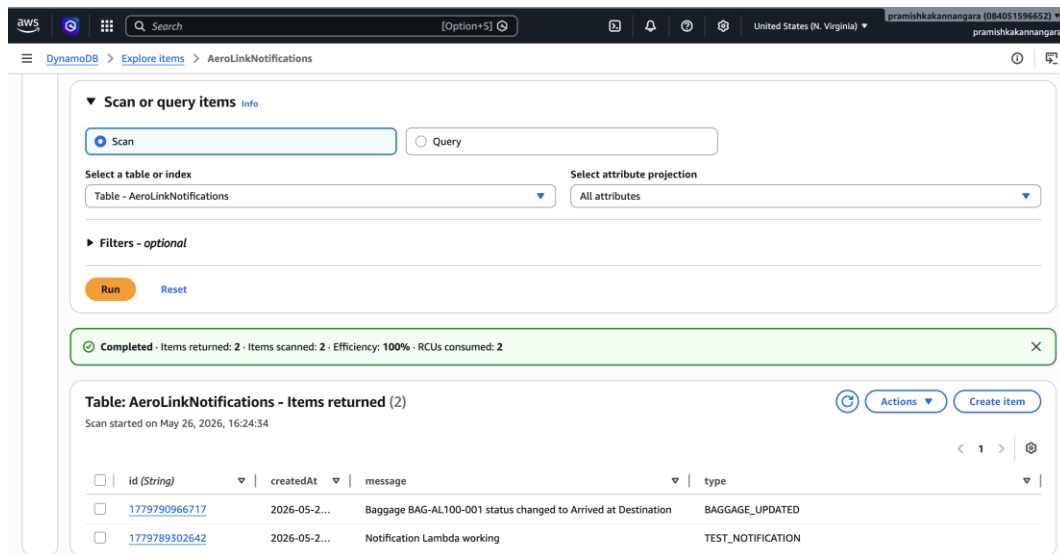


Figure 29: Notification records automatically generated after EventBridge event processing.

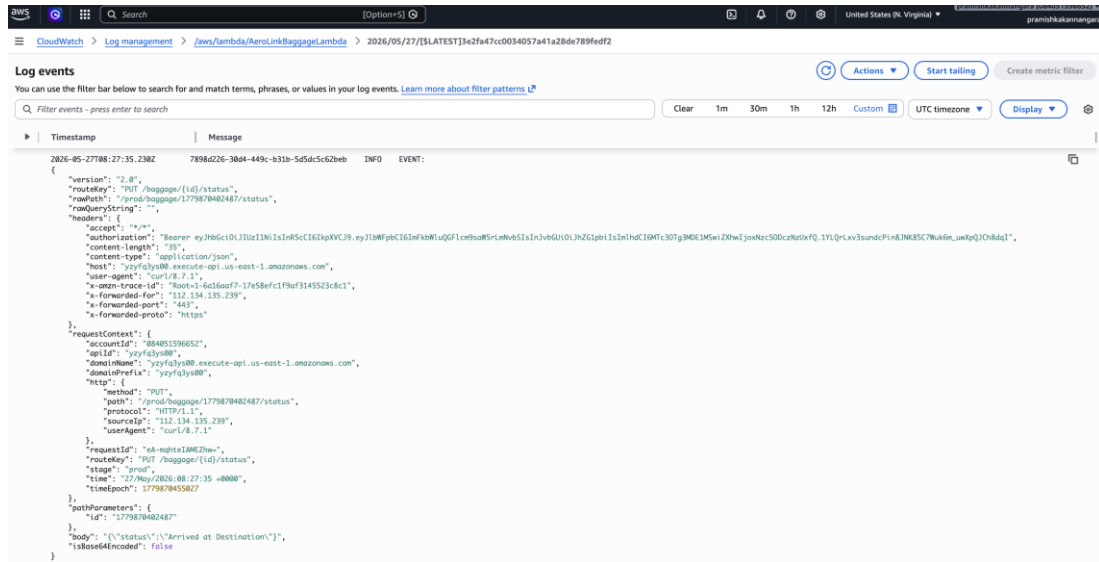


Figure 30: CloudWatch logs showing successful processing of EventBridge events by the notification service.

Figure 32: Protected API Documentation

3.8.2 HTTPS and API Gateway Security Controls

Apart from raising the security level of the AeroLink architecture, the client apps' and microservices' interactions are established using the Amazon API Gateway via the use of the HTTPS protocol. The usage of the HTTPS protocol makes sure that all communications are done in a networked manner with encryption over the TLS protocol and thus no data interceptions are possible.

Furthermore, the API Gateway acts as another layer of security as every request sent has to be done through the API Gateway prior to reaching the backend services. In particular, every request requires the correct authorization header with the proper JWT token for accessing the secured services.

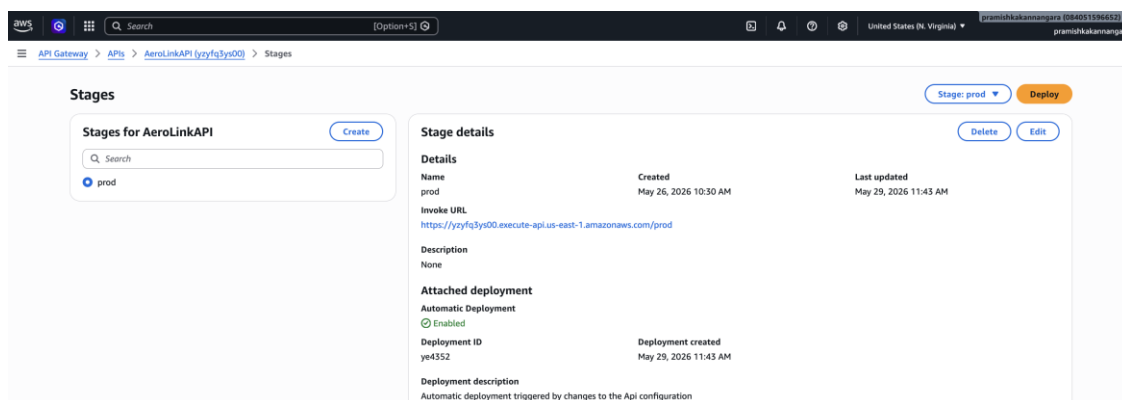


Figure 33: Amazon API Gateway provides secure HTTPS endpoints for communication between AeroLink microservices.

3.9 Frontend User Interface Implementation

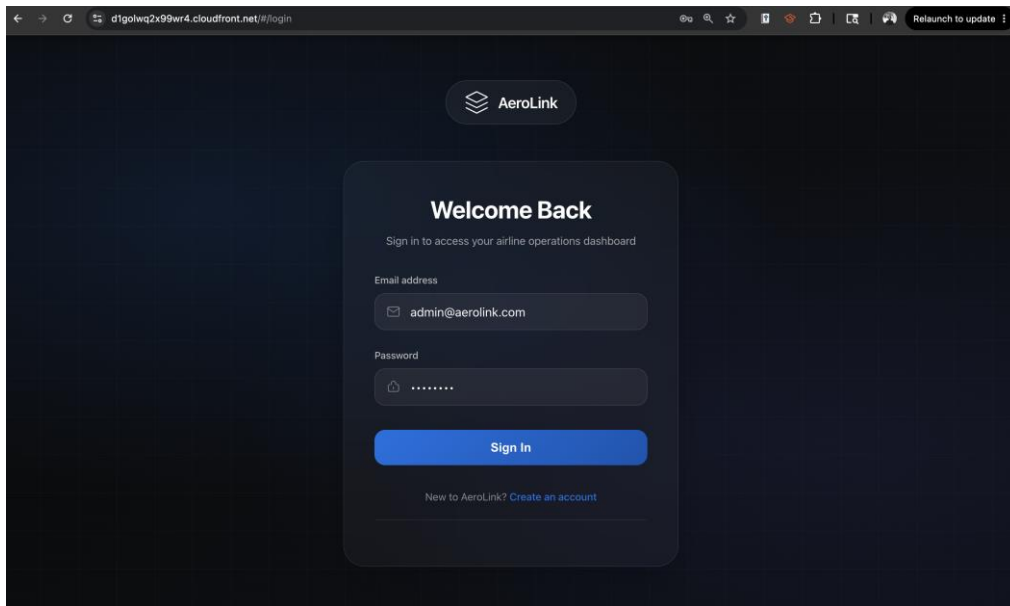
The AeroLink frontend was made using React.js, JavaScript, HTML5 and CSS3. It is a web application that talks to cloud-hosted backend services through Amazon API Gateway using APIs.

The user interface has parts, for passengers, staff and administrators. This means that AeroLink can do lots of things like let users log in manage flights, book passengers, process payments do check-in operations, track baggage manage airport events send notifications and control who can do what.

AeroLink uses JWT tokens to keep protected things. It also uses Axios to talk to the backend services that are hosted on AWS Lambda and Amazon ECS Fargate.

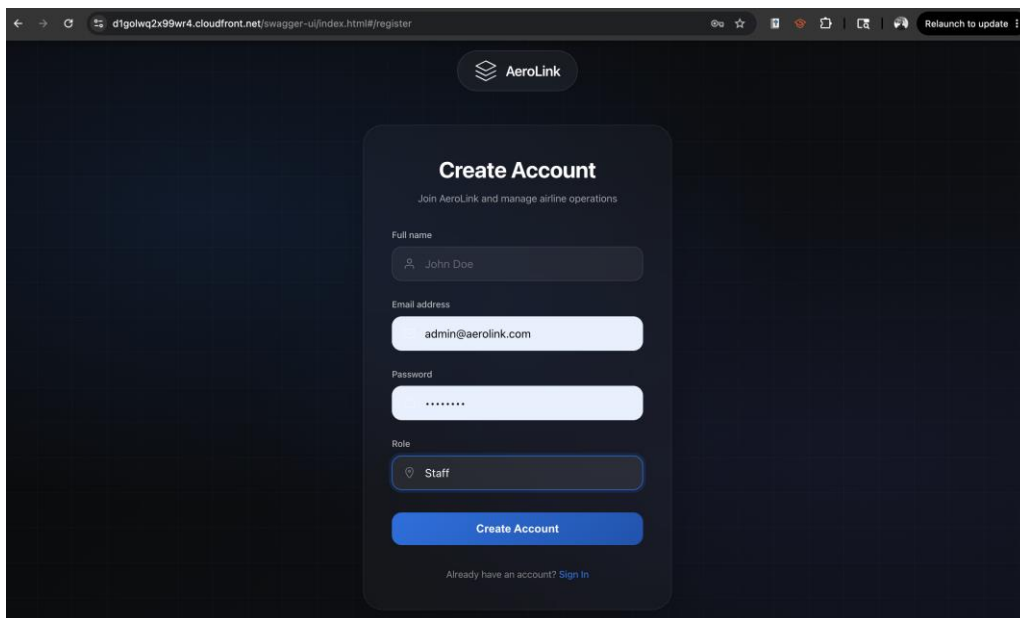
The interface that was made shows that the presentation layer works with the cloud-native microservices architecture. It gives an environment for doing all the work that the AeroLink airline

management platform needs. The AeroLink frontend is a part of the AeroLink airline management platform and it helps AeroLink do its job.



The image shows a web browser window displaying the AeroLink login interface. The browser's address bar shows the URL `d1golwq2x99wr4.cloudfront.net/#/login`. The AeroLink logo is at the top center. The main heading is "Welcome Back" with the subtext "Sign in to access your airline operations dashboard". Below this are two input fields: "Email address" with the value `admin@aerolink.com` and "Password" with masked characters. A blue "Sign In" button is positioned below the password field. At the bottom, there is a link that says "New to AeroLink? Create an account".

Figure 34:User Login Interface



The image shows a web browser window displaying the AeroLink registration interface. The browser's address bar shows the URL `d1golwq2x99wr4.cloudfront.net/swagger-ui/index.html#/register`. The AeroLink logo is at the top center. The main heading is "Create Account" with the subtext "Join AeroLink and manage airline operations". Below this are four input fields: "Full name" with the value `John Doe`, "Email address" with the value `admin@aerolink.com`, "Password" with masked characters, and "Role" with a dropdown menu showing "Staff". A blue "Create Account" button is positioned below the role field. At the bottom, there is a link that says "Already have an account? Sign In".

Figure 35:User Register Interface

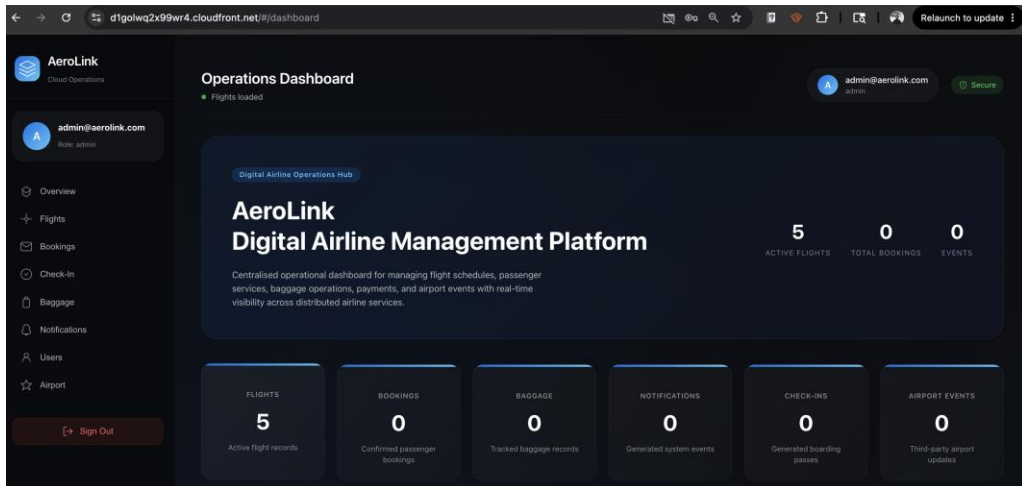


Figure 36:Dashboard Overview

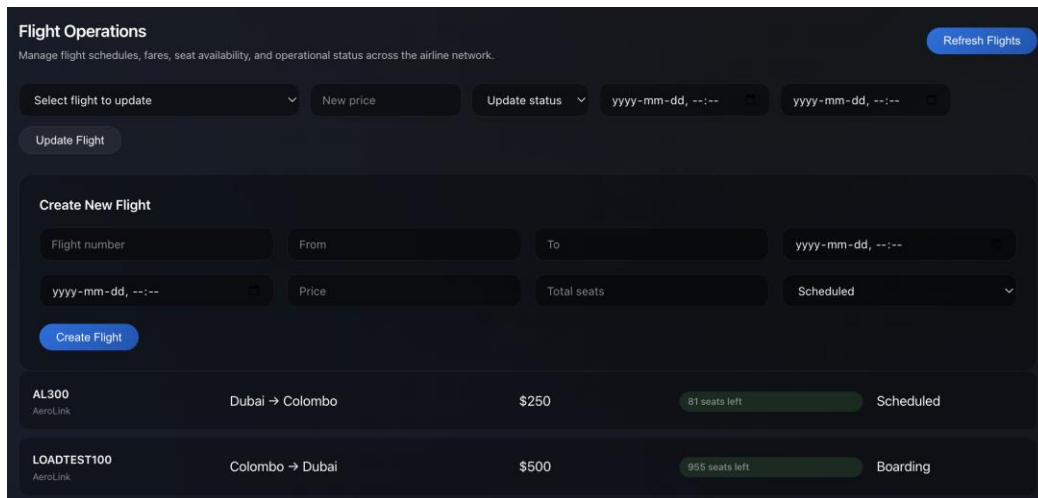


Figure 37:Flight Management Interface

Passenger Booking

Create and manage passenger reservations with automated seat allocation, fare calculation, payment verification, and inventory updates.

Select Flight

Choose the flight you want to book before making the payment.

Select a flight

1

Payment Details

Test card: 4111111111111111, Expiry 12/30, CVV 123

Cardholder name

Card number

MM/YY

CVV

Payment Approved

Provider: Simulated Stripe Provider

Status: PAYMENT_APPROVED

Transaction Ref: TXN-1780572450389

Card Last 4: **** 6209

Create Booking

Load Bookings

Search bookings...

Booking #1779867129600

LOADTEST100 AeroLink, Colombo → Dubai, \$500

admin@aerolink.com booked 1 seat(s)

Booked at: 27/05/2026, 13:02:09

Confirmed

Booking #1779867128700

LOADTEST100 AeroLink, Colombo → Dubai, \$500

admin@aerolink.com booked 1 seat(s)

Booked at: 27/05/2026, 13:02:08

Confirmed

Booking #1779788216239

AL100 AeroLink, Colombo → Dubai, \$450

Pramishka booked 2 seat(s)

Booked at: 26/05/2026, 15:06:56

Confirmed

Booking #1779867132517

Figure 38:Passenger Booking and Payment Interface

Passenger Check-in

Process passenger check-ins and generate boarding passes with assigned seats, gate information, and travel records.

Select booking for check-in

Check In Passenger

Load Check-ins

BP-1780034275231

Passenger: Pramishka

Booking ID: #TEST-BOOKING-001

Flight ID: 1

Seat: A24

Gate: G12

CHECKED_IN

BP-1780034886969

Passenger: pramishka@gmail.com

Booking ID: #178003333030

Flight ID: 1779866900993

Seat: A23

Gate: G12

CHECKED_IN

BP-1780485483281

Passenger: Pramishka

Booking ID: #TEST-BOOKING-001

Flight ID: 1

Search baggage...

BAG-AL100-001

Passenger: Pramishka

Booking ID: #1

Checked in

BAG-AL100-001

Passenger: Pramishka

Booking ID: #1

Arrived at Destination

Figure 39:Passenger Check in Interface and baggage tracking

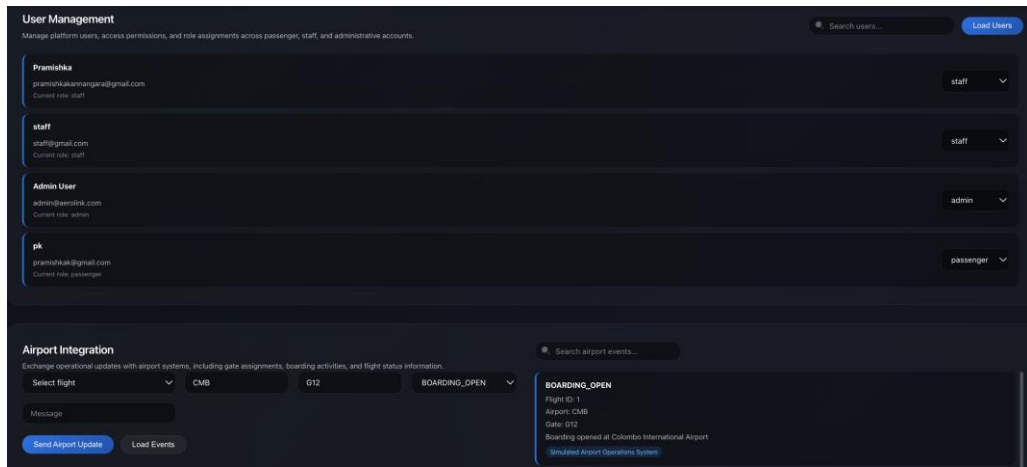


Figure 40: Airport Integration, Event Management Interface and User management

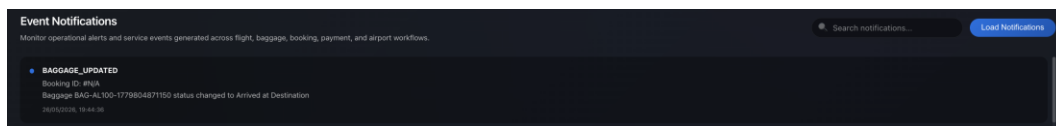


Figure 41: Notification section

4 Data Security, Compliance and Consistency

4.1 Encryption at Rest

Encryption of data is achieved using the encrypt feature provided by AWS in connection with the data stored in the Amazon DynamoDB. Data at rest encryption can be done for DynamoDB tables by using the encryption keys provided by AWS. Sensitive configuration parameters can also be encrypted using AWS Lambda environment variables. The purpose here is to ensure that data such as customer details, bookings, baggage details, and transactions are safe.

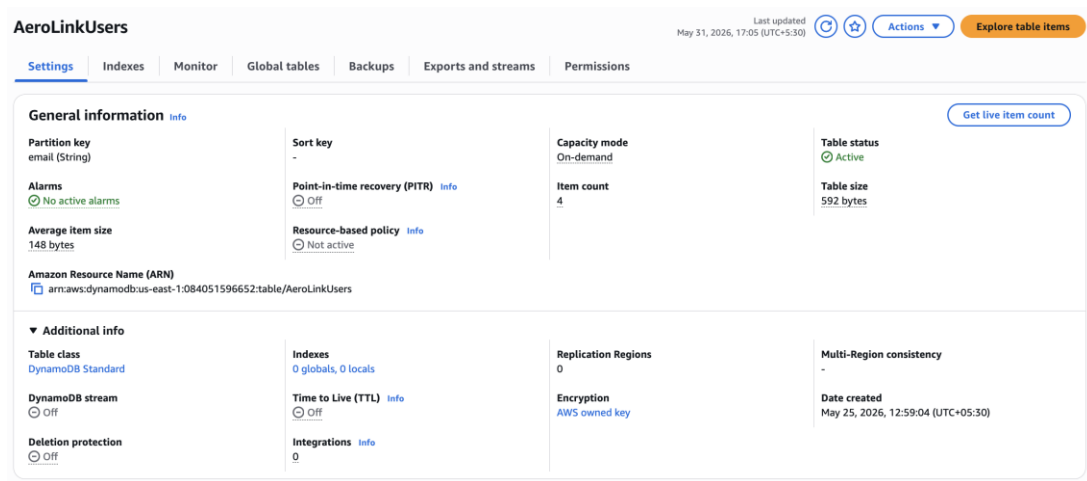


Figure 42: DynamoDB encryption at rest using AWS-owned encryption keys.

4.2 Encryption in Transit

HTTPS encryption protocol, together with TLS version 1.2 or higher, is used for security purposes when all the AeroLink communication channels work. All SSL/TLS will be provided by Amazon API Gateway, which will use SSL/TLS endpointing. In the end, all the confidential data will be protected with encryption during transmission over the network.

Stage details

Details

Name	Created
prod	May 26, 2026 10:30 AM
Invoke URL	
https://zyfq3ys00.execute-api.us-east-1.amazonaws.com/prod	

Figure 43: Secure HTTPS endpoint provided by Amazon API Gateway using TLS encryption.

4.3 Authentication and Authorization

The methods of authentication and authorization using JWT and RBAC are implemented in the case of AeroLink platforms. After login by the user, he gets a token from which he needs to use when making any future API calls. On the contrary, RBAC denies access to certain types of data depending on the user role.



Figure 44: JWT token issued after successful user authentication.

4.4 GDPR Compliance Considerations

Principles of GDPR have been observed during the design of the AeroLink platform. Collection of only required data of the customers helps in abiding by the data minimization principle. Customer consent has been obtained before the collection of any personal data from the customer, and the data is stored only if required by the customer. Additionally, an audit trail has been maintained to record system logs.

4.5 PCI-DSS Compliance Considerations

Since AeroLink handles transactions, there are also a few aspects of PCI-DSS that can be found in its structure. Transactions data travels through the secure connection using HTTPS protocol, while the data about cards is also encrypted before storage or processing. Additionally, the Payment Service is isolated from other services for increased security reasons.



Figure 45: Secure payment authorization process supporting PCI-DSS compliance requirements.

4.6 Data Consistency Strategy

As the AeroLink architecture is a distributed architecture, achieving absolute consistency in the operations performed by all the services would lead to a reduction in performance and scalability. This is the reason why the system utilizes eventual consistency where there is no need for immediate consistency. DynamoDB facilitates this with asynchronous updates from one service to another and within different regions. Business events can be communicated using Amazon EventBridge.

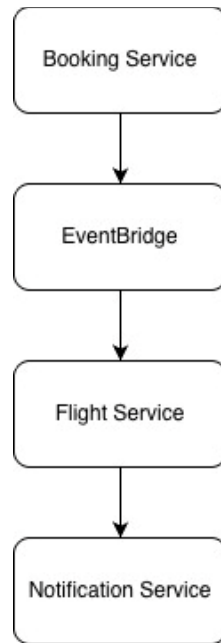


Figure 46: Eventual Consistency Model using EventBridge

4.7 Saga Pattern Justification

The Saga Pattern is used by AeroLink to ensure coordination in managing distributed transactions among multiple microservices. The services don't use a transaction in the database but instead execute their operation individually while triggering an event for the subsequent service in the process flow.

In case there is a problem in the payment process, a compensating transaction is made to rollback the booking process and ensure that the seats are available again.

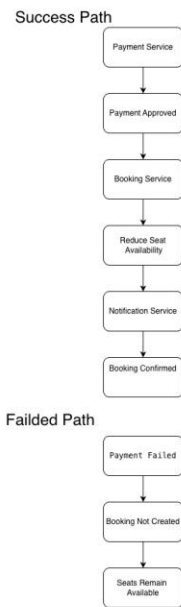


Figure 47: Saga Transaction Flow

5 Real-Time Data Synchronisation

5.1 Flight Seat Availability Updates

On the other hand, from the perspective of availability of seats, the process is complete once the passenger finishes booking. Booking is the act of making the payment and updating the status of seat availability for that specific flight. This ensures that no overbooking occurs on that particular flight. To put it in another way, the moment any passenger finishes booking, they get to know the seat availability status for that specific flight.

AL100 AeroLink	Colombo → Dubai	\$450	89 seats left	Scheduled
-------------------	-----------------	-------	---------------	-----------

Figure 48: Booking creation screen showing selected flight and number of seats.

Completed · Items returned: 1 · Items scanned: 1 · Efficiency: 100% · RCUs consumed: 2

Table: AeroLinkFlights - Items returned (1)									
Scan started on May 26, 2026, 15:13:25									
☐	Id (String)	availableSeats	departureTime	flightNumber	from	price	status	to	totalSeats
☐	1	98	2026-06-10T08:00:00	AL100	Colombo	450	Scheduled	Dubai	100

Figure 49: Flight record after booking showing reduced available seats.

5.2 Baggage Status Synchronization

Syncing of the status of the luggage will be done via the Amazon EventBridge tool. Once there is any update made on the data related to luggage from the Baggage Service end, there occurs an event that goes by the name "BAGGAGE_UPDATED," and it reaches the Notification Service through EventBridge.

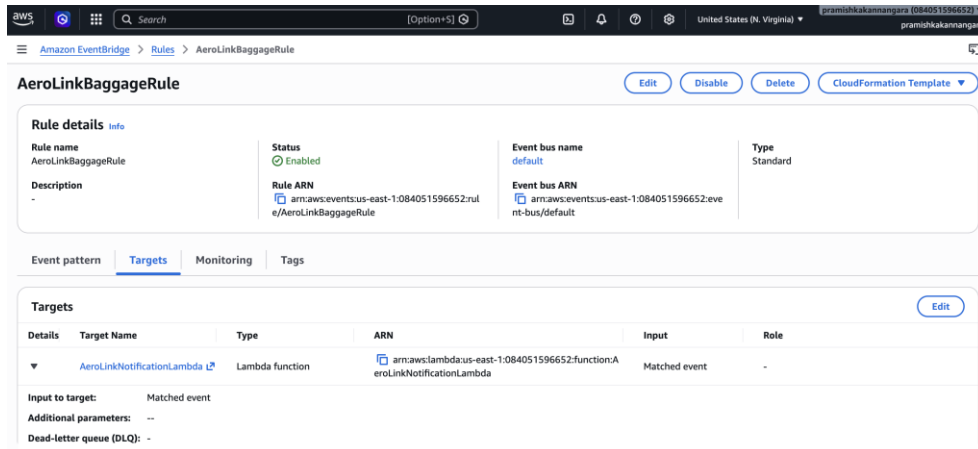


Figure 50: EventBridge rule routing baggage update events to the Notification Service.

Table: AeroLinkNotifications - Items returned (17)
Scan started on June 01, 2026, 11:23:57

id (String)	bookin...	createdAt	message	type	userEmail
1779804876468		2026-05-2...	Baggage BAG-AL100-177980487...	BAGGAGE_UPDATED	
1779819693375	1	2026-05-2...	Baggage BAG-AL100-177980688...	BAGGAGE_UPDATED	pramishkak@gmail.com
1779820759436	17798178...	2026-05-2...	Baggage BAG-1779817854095-1...	BAGGAGE_UPDATED	pramishkak@gmail.com

Figure 51: DynamoDB notification record generated after baggage status update.

5.3 Flight Schedule Updates

The update in relation to flight schedules shall be carried out using Flight Service. This can be done by the Admin or the staff member in their respective dashboard. After the update has been made, the system shall automatically reload the updated flight details on the frontend.

Table: AeroLinkFlights - Items returned (5)
Scan started on June 01, 2026, 11:27:46

id (String)	departureTime	flightNumber	from	price	status	to	totalSeats	update...	updatedBy
1779854972247	2026-05-28T09:38	AL300	Dubai	500	Scheduled	Colombo	85	2026-05-...	
1779866900993	2026-08-01T08:00:00	LOADTEST100	Colombo	500	Boarding	Dubai	1000	2026-05-...	admin@aerolink.com
1	2026-06-10T08:00:00	AL100	Colombo	450	Scheduled	Dubai	100		
1779854033186	2026-07-01T09:00:00	AL200	Colombo	670	Scheduled	Singapore	120	2026-05-...	admin@aerolink.com
1779860574804	2026-06-05T11:11	AI150	New Zealand	990	Cancelled	Colombo	110	2026-05-...	staff@gmail.com

Figure 52: Updated flight record in DynamoDB.

5.4 Pricing Updates

Flight prices can also be modified via Flight Service. Any modification to the price by the admin or staff would be stored in DynamoDB, and the modified price would be fetched by the users on

refreshing the flight details list. In the real-world application, this price change can be pushed via EventBridge service, allowing Booking and Notification services to know about the change.

Figure 5.7: Flight pricing update form in the AeroLink dashboard.

Table: AeroLinkFlights - Items returned (5)
Scan started on June 01, 2026, 11:36:12

Actions

Create item

< 1 >

☐	id (String)	departureTime	flightNumber	from	price	status	to	totalSeats	update...	updatedBy
☐	1779854972247	2026-05-28T09:38	AL300	Dubai	250	Scheduled	Colombo	85	2026-06-...	admin@aerolink.com

Figure 53: DynamoDB flight table showing updated price.

5.5 Synchronization Results

This approach of synchronization was tested with booking creation, baggage status changes, flight scheduling changes, and pricing changes through the AeroLink control panel and seeing how fast they reflect on their respective services. As evident from these examples, the necessary updates are reflected on services almost instantly. The changes in the status of the baggage are event-based using the AWS EventBridge service, while the other changes use API calls and front-end refreshing.

1: Real Time Synchronisation Scenarios and Expected Outcomes

Event	Services	Time delay
Baggage status update	EventBridge and Notification Service	< 1 second
Booking and seat update	Booking Service and Flight Service refresh	< 1 second
Flight schedule update	Flight Service API update	< 1 second

6 Fault Tolerance and Resilience

6.1 Retry Policies

The reason why AeroLink has high tolerance to faults is due to the implementation of retry mechanisms at different levels of the solution stack. The AWS Lambda feature allows for an automatic retry for any failed asynchronous calls. Likewise, Amazon EventBridge can retry failed event deliveries. As long as the event has not been considered a failure, it will retry. Similarly, API request from the frontend layer is being retried.

```
frontend > src > pages > Dashboard.jsx > Dashboard > apiRequest
8   function Dashboard() {
  / 4
75   const apiRequest = async (requestFunction, retries = 2) => {
76     try {
77       return await requestFunction();
```

Figure 54: Frontend retry logic used for temporary API failures.

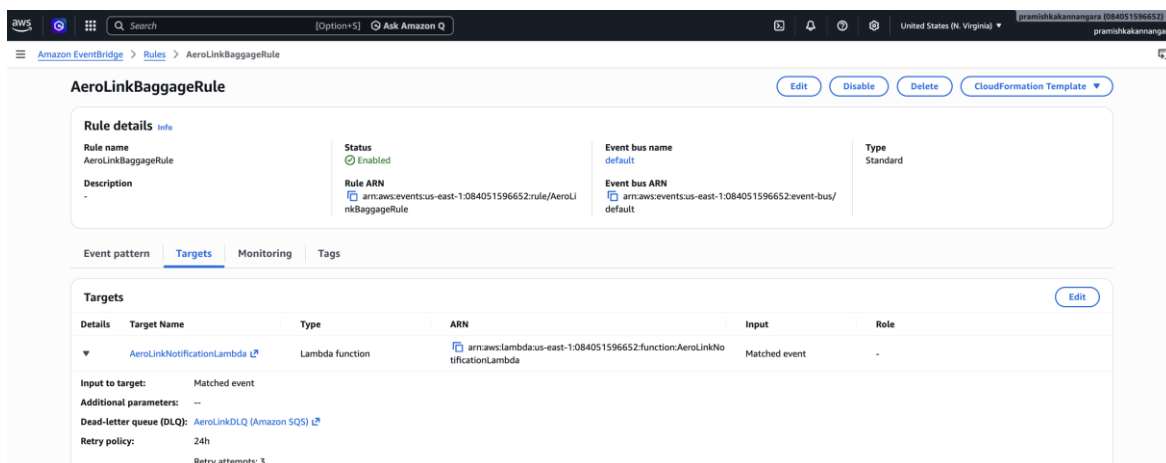


Figure 55: EventBridge rule used for event delivery retry handling.

6.2 Dead Letter Queues

Dead Letter Queue may also be considered as a means of storing those events that could not be processed despite retrying. According to AeroLink guidelines, this may be done especially for critical events such as baggage information updates, booking confirmation notifications, and other similar events. Failed messages can later be inspected and reprocessed without losing operational data.

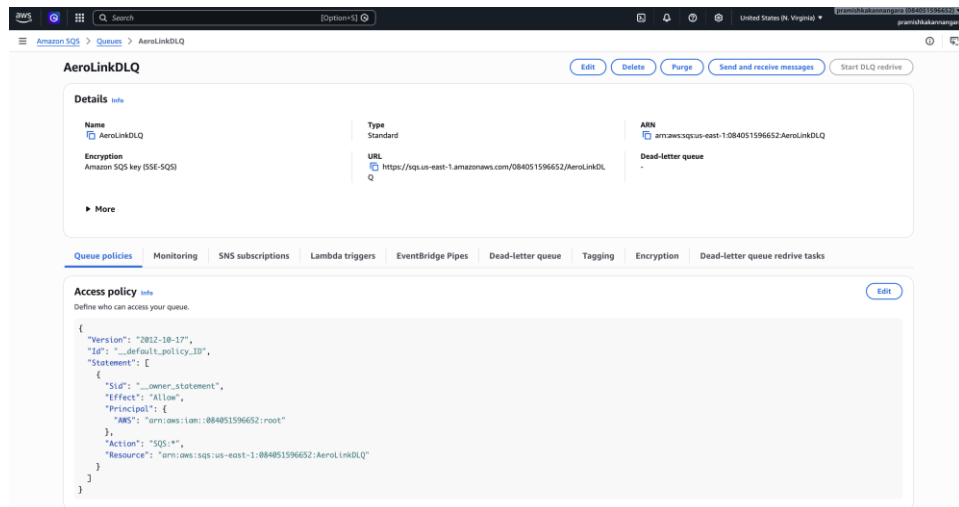


Figure 56: Amazon SQS Dead Letter Queue configuration for failed messages.

6.3 Circuit Breaker Pattern

The use of the Circuit Breaker design pattern is conceptual in nature, and its purpose is to guard against invoking a failed service again and again. For instance, when the Payment Service fails to respond, it is necessary that the Booking Service not bombard the Payment Service with payment requests and throw a specific failure message.

```
frontend > src > pages > Dashboard.jsx > Dashboard > [circuitBreakerRequest]
8 function Dashboard() {
74
75   const failureCountRef = useRef(0);
76   const circuitOpenUntilRef = useRef(null);
77
78   const circuitBreakerRequest = async (requestFunction) => {
79     const now = Date.now();
80
81     if (circuitOpenUntilRef.current && now < circuitOpenUntilRef.current) {
82       throw new Error("Circuit breaker is open. Service temporarily unavailable.");
83     }
84
85     try {
86       const result = await requestFunction();
87       failureCountRef.current = 0;
88       circuitOpenUntilRef.current = null;
89       return result;
90     } catch (error) {
91       failureCountRef.current += 1;
92
93       if (failureCountRef.current >= 3) {
94         circuitOpenUntilRef.current = Date.now() + 30000;
95       }
96
97       throw error;
98     }
99   };
}
```

Figure 57: Circuit breaker logic used to stop repeated calls to a temporarily failing service.

6.4 Load Balancing

Load balancing is used to distribute the load across various backend services and avoid an overload on one single service. This would be achieved by the following proposed architecture of production where the traffic gets routed via CloudFront, API Gateway, and other load balancing services before reaching the ECS Fargate services.



Figure 58: Amazon CloudFront distribution improving availability and traffic distribution.

6.5 Auto Scaling

The use of Fargate ECS is done within the architecture of AeroLink microservices through containerization. The advantage of this is that each service can be scaled separately depending on its load. In other words, if the load is high, such as during booking hours, then more tasks can be created for some services without scaling the entire application. This also happens with the automatic scaling of database loads using DynamoDB on-demand.

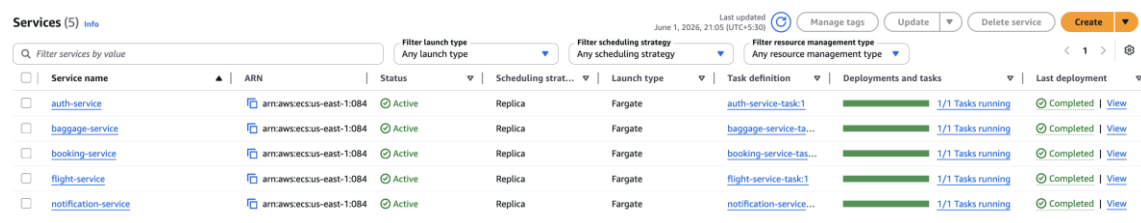


Figure 59: ECS Fargate services supporting independent scaling of AeroLink microservices.

General information Info

Partition key
id (String)

Alarms
✔ No active alarms

Average item size
213.5 bytes

Amazon Resource Name (ARN)
arn:aws:dynamodb:us-east-1:084051596652:table/AeroLinkCheckIns

Sort key
-

Point-in-time recovery (PITR) Info
⊖ Off

Resource-based policy Info
⊖ Not active

Capacity mode
On-demand

Item count
2

Figure 60: DynamoDB on demand capacity supporting automatic database scaling.

6.6 Multi-AZ Deployment

High availability is facilitated by implementing the services using various Availability Zones, such that if one Availability Zone fails, the platform can continue to be run from the other zones. High availability can thus be achieved using ECS Fargate, DynamoDB, and API Gateway.

baggage-service-task:1	Unknown	4 days ago	ecs-ovc/71675849950...	4 days ago	service:baggage-service	-	Fargate	1.4.0	.25 vCPU	.5 GiB	us-east-1e
flight-service-task:1	Unknown	4 days ago	ecs-ovc/55372467959...	4 days ago	service:flight-service	-	Fargate	1.4.0	.25 vCPU	.5 GiB	us-east-1a

Figure 61:ECS services deployment across multiple Availability Zones to improve availability.

6.7 Disaster Recovery

In the case of disaster recovery strategy, the main production region will be us-east-1, and the second one where the disaster recovery should happen is eu-west-1. In case the region fails, then routing of the network traffic will be done using Route 53's Failover Routing method. Using DynamoDB global tables for the replication of the data along with other methods, the services can again become available.

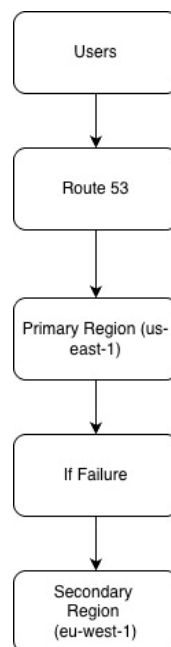


Figure 62:Proposed Disaster Recovery Strategy

7 Performance and Scalability Testing

7.1 Testing Environment

The performance testing was done in the production environment of the AeroLink cloud system with K6. The communication process between the front-end and back-end is facilitated by using the Amazon API Gateway. AWS Lambda and ECS Fargate microservices are used for implementing the back-end processes. All data related to the operations such as flights, booking, baggage, notifications, and payments are stored in the DynamoDB database.

7.2 Load Testing Methodology

Tests were carried out through the use of Grafana k6 testing application. In the process of performing these tests, the test scripts utilized HTTPS protocol to test the APIs, such as flights, booking, and notification APIs. In case of the booking load tests, the tests were conducted using two virtual users in 10 seconds. However, the general load testing was conducted by running 20 virtual users in 30 seconds. The following are some of the measures used during performance tests: request latency, request per second, and failed requests.

```
performance-tests > load-test.js > default > headers
1  import http from 'k6/http';
2  import { sleep } from 'k6';
3
4  export const options = {
5    vus: 20,
6    duration: '30s',
7  };
8
9  const BASE_URL = 'https://yzfq3ys00.execute-api.us-east-1.amazonaws.com/prod';
10
11 export default function () {
12   http.get(`${BASE_URL}/flights`);
13
14   http.get(`${BASE_URL}/notifications`, {
15     headers: {
16       Authorization: 'Bearer TEST_TOKEN'
17     }
18   });
19 };
20
21 sleep(1);
22 }
```

Figure 63:k6 load testing script

7.3 Load Testing Results

The general load test involved a total of 660 requests by twenty virtual users over 30 seconds with an average response time of 406.2 milliseconds and average throughput of 21.02 requests/second. The failed requests accounted for 5.00%, and a total of 33 requests failed from the total 660 requests sent. During the booking load test, there were a total of 41 requests and 0% failure rate. Average time taken per HTTP request in milliseconds was 454.57 while the average throughput

was 4.07 requests/second. It is evident that the booking endpoint could deal with the concurrent requests unlike others where failures occurred due to endpoint security.

```
pramishkannagaram2@192 performance-tests % k6 run load-test.js

Grafana
K6

execution: local
script: load-test.js
output: -

scenarios: (100.00%) 1 scenario, 20 max VUs, 1m0s max duration (incl. graceful stop):
* default: 20 looping VUs for 30s (gracefulStop: 30s)

■ TOTAL RESULTS

HTTP
http_req_duration.....: avg=406.2ms min=274.95ms med=350.66ms max=1.95s p(90)=431.83ms p(95)=478.66ms
{ expected_response:true }...: avg=408.36ms min=274.95ms med=350.54ms max=1.95s p(90)=434.6ms p(95)=480.92ms
http_req_failed.....: 5.00% 33 out of 660
http_reqs.....: 660 21.018886/s

EXECUTION
iteration_duration.....: avg=1.85s min=1.56s med=1.71s max=5.39s p(90)=1.88s p(95)=2.65s
iterations.....: 330 10.509443/s
vus.....: 3 min=3 max=20
vus_max.....: 20 min=20 max=20

NETWORK
data_received.....: 922 kB 29 kB/s
data_sent.....: 66 kB 2.1 kB/s

running (0m31.4s), 00/20 VUs, 330 complete and 0 interrupted iterations
default ✓ [=====] 20 VUs 30s
pramishkannagaram2@192 performance-tests %
```

Figure 64:load-test.js result screenshot

```
pramishkannagaram2@192 performance-tests % k6 run booking-test.js

Grafana
K6

execution: local
script: booking-test.js
output: -

scenarios: (100.00%) 1 scenario, 2 max VUs, 40s max duration (incl. graceful stop):
* default: 2 looping VUs for 10s (gracefulStop: 30s)

■ TOTAL RESULTS

HTTP
http_req_duration.....: avg=454.57ms min=336.55ms med=378.97ms max=1.79s p(90)=470.39ms p(95)=529.55ms
{ expected_response:true }...: avg=454.57ms min=336.55ms med=378.97ms max=1.79s p(90)=470.39ms p(95)=529.55ms
http_req_failed.....: 0.00% 0 out of 41
http_reqs.....: 41 4.074057/s

EXECUTION
iteration_duration.....: avg=489.42ms min=336.77ms med=379.1ms max=2.5s p(90)=470.62ms p(95)=529.8ms
iterations.....: 41 4.074057/s
vus.....: 2 min=2 max=2
vus_max.....: 2 min=2 max=2

NETWORK
data_received.....: 26 kB 2.6 kB/s
data_sent.....: 11 kB 1.1 kB/s

running (10.1s), 0/2 VUs, 41 complete and 0 interrupted iterations
default ✓ [=====] 2 VUs 10s
```

Figure 65:booking-test.js result screenshot

2:Load Testing Results for AeroLink APIs

Test	Virtual users	Request	Average latency	Throughput	Error rate
General load test	20	660	406.2 ms	4.07 req/s	0%
Booking load test	2	41	454.57 ms	4.07 req/s	0%

7.4 Stress Testing Results

For performing stress testing, the number of virtual users was gradually increased until 100. The stress test was able to complete 4005 transactions with an average latency of 339.51 ms and throughput of around 49.73 transactions per second. The error percentage also rose up to 5.66%, thereby indicating the availability of the application and its ability to process transactions efficiently.

```
performance-tests >  stress-test.js >  default
1  import http from 'k6/http';
2  import { sleep } from 'k6';
3
4  export const options = {
5    stages: [
6      { duration: '20s', target: 20 },
7      { duration: '20s', target: 50 },
8      { duration: '20s', target: 100 },
9      { duration: '20s', target: 0 },
10   ],
11 };
12
13 const BASE_URL = 'https://zyzfq3ys00.execute-api.us-east-1.amazonaws.com/prod';
14
15 export default function () {
16   http.get(`${BASE_URL}/flights`);
17
18   sleep(0.5);
19 }
20
```

Figure 66:k6 Stress testing script

```
pramishkakannagaram2@192 performance-tests % k6 run stress-test.js

Grafana
K6

execution: local
script: stress-test.js
output: -

scenarios: (100.00%) 1 scenario, 100 max VUs, 1m50s max duration (incl. graceful stop):
* default: Up to 100 looping VUs for 1m20s over 4 stages (gracefulRampDown: 30s, gracefulStop: 30s)

■ TOTAL RESULTS

HTTP
http_req_duration.....: avg=339.51ms min=256.32ms med=324.22ms max=3.48s p(90)=386.27ms p(95)=415.57ms
{ expected_response:true }...: avg=341.24ms min=266.1ms med=325.45ms max=3.48s p(90)=388.09ms p(95)=417.69ms
http_req_failed.....: 5.66% 227 out of 4005
http_reqs.....: 4005 49.734599/s

EXECUTION
iteration_duration.....: avg=855.67ms min=756.51ms med=825.88ms max=3.98s p(90)=897.32ms p(95)=942.92ms
iterations.....: 4005 49.734599/s
vus.....: 2 min=1 max=99
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 6.2 MB 76 kB/s
data_sent.....: 356 kB 4.4 kB/s

running (1m20.5s), 000/100 VUs, 4005 complete and 0 interrupted iterations
default ✓ [=====] 000/100 VUs 1m20s
```

Figure 67: stress-test.js result screenshot

3: Stress Testing Results and System Limits

Test	Virtual users	Request	Average latency	Throughput	Error rate
Stress test	100	4005	339.51ms	49.73 req/s	5.66%

7.5 Throughput Analysis

The throughput results indicate that AeroLink is able to handle more requests as the number of requests increases. The test on booking load gave about 4.07 requests per second, whereas the test on general load resulted in about 21.02 requests per second. With the use of stress testing, the level of throughput was further enhanced, resulting in about 49.73 requests per second. This clearly indicates that AeroLink is able to cope up with growing traffic.

7.6 Error Rate Analysis

In booking load testing, there were no failed requests at all. This shows that the performance of the system under low loads was satisfactory. The overall failure rates in general load testing was 5.00%, whereas in stress testing it reached 5.66%. This could be attributed to various reasons such as authentication issues, request resource limitations, and backend services limitations. So far, the

performance of the system under testing was quite satisfactory. However, the system requires optimization and scalability improvements.

7.7 Performance Improvements

There are certain changes that would enhance the AeroLink's ability to perform efficiently and scale effectively. For instance, caching in the API Gateway and CloudFront would help reduce repeated requests made to the backend services. The ECS Service Auto Scaling should also be able to trigger automatic launching of more containers when the demand is at its peak. Although the DynamoDB on-demand capacity helps in variable database workloads, a solution to transactional workload volumes may include read replicas or Amazon Aurora for relational booking data. Additional optimisation can also be achieved through request throttling, improved JWT handling, and better error handling in API endpoints.

8 Monitoring and Observability

8.1 AWS CloudWatch Monitoring

CloudWatch from AWS was used as the tool for monitoring the distributed system. The tool is responsible for monitoring the performance of such AWS services as Lambda, API Gateway, and ECS Fargate services within AeroLink. Therefore, it is possible to monitor the entire distributed system in real-time mode. Such data is obtained through CloudWatch Dashboards, including function invocations, execution time, errors, and other details. Due to the use of the centralized monitoring method, any issues can be identified before being realized by end users.

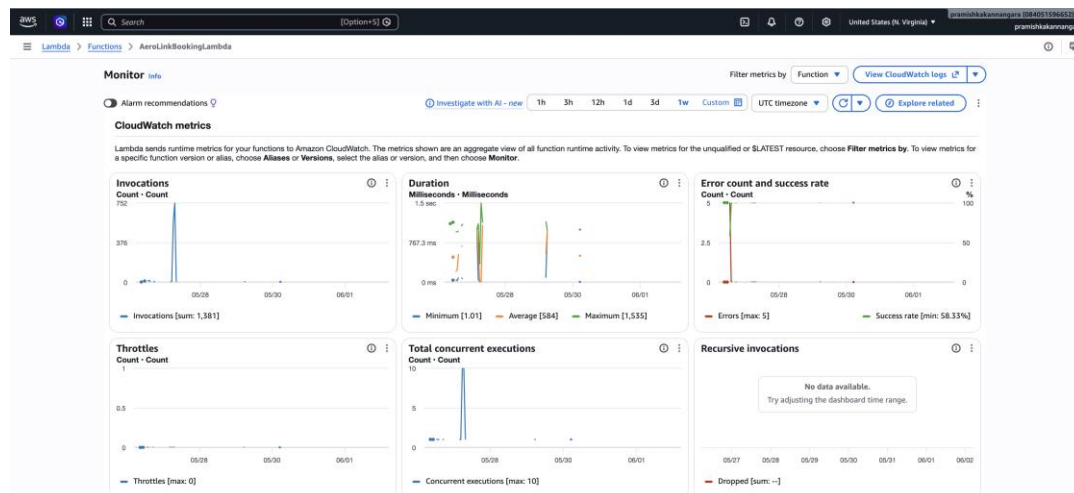


Figure 68: CloudWatch dashboard displaying Lambda performance metrics.

8.2 Application Logging

At the application level, logging has been done using AWS CloudWatch Logs. All the services deployed on Lambda automatically write their execution logs and errors into log groups of CloudWatch. The logs give clear insight into the services' behaviour. During the tests, logs were also used to determine whether authentication, booking, baggage, and notifications processes had been successfully completed.

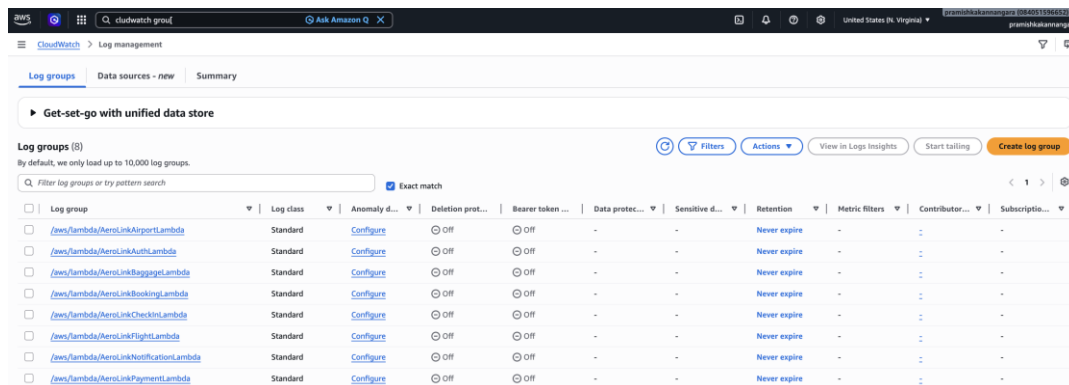


Figure 69: CloudWatch log groups for AeroLink microservices.



Figure 70: Lambda execution logs captured in CloudWatch.

8.3 Metrics Collection

The CloudWatch metrics were employed for gathering statistics about the system performance. The measurements included in the CloudWatch metrics are: number of invocations, average duration, failure rate, and number of throttled requests. We receive a lot of information about the system performance with the help of these instruments. The combination of CloudWatch metrics and the findings from the k6 load testing will be highly important for assessing the system performance under heavy load conditions. Such indicators help in capacity planning and performance optimization.

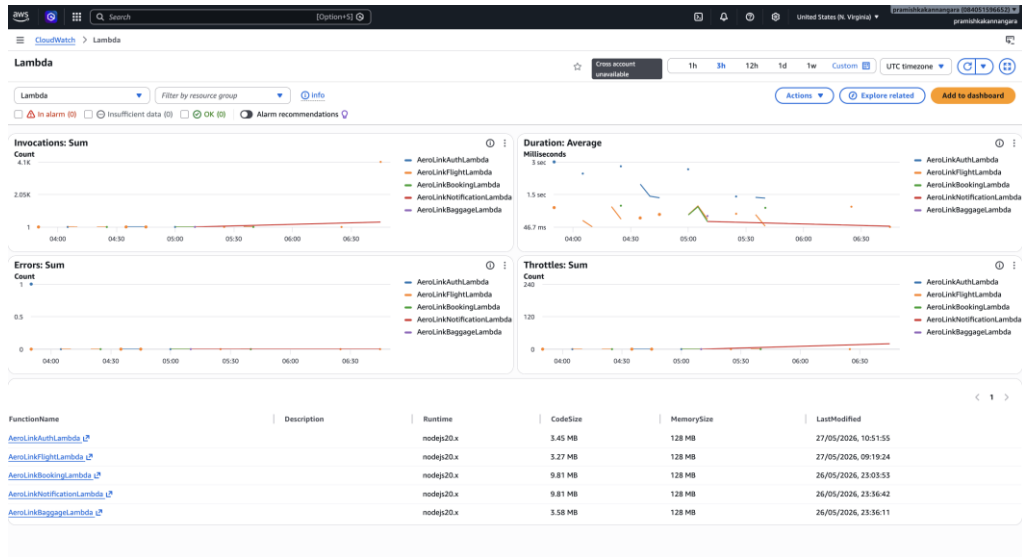


Figure 71: CloudWatch metrics collected during performance testing.

8.4 Distributed Tracing

The development of AWS X-Ray was prompted by the need to track the operation of the Aerolink microservice architecture using the concept of distributed tracing. With AWS X-Ray, it will become easier to trace request processing from one AWS component to another, and thus monitor the whole process of request processing. Using AWS X-Ray service maps, the way in which different cloud components such as API gateway and lambdas interact with each other during the process can be understood.

Use of AWS X-Ray in distributed tracing will be very helpful in monitoring passenger booking requests, baggage requests, notifications, and payment requests in Aerolink. It makes observing more effective since there will be better understanding of how various services depend on each other.

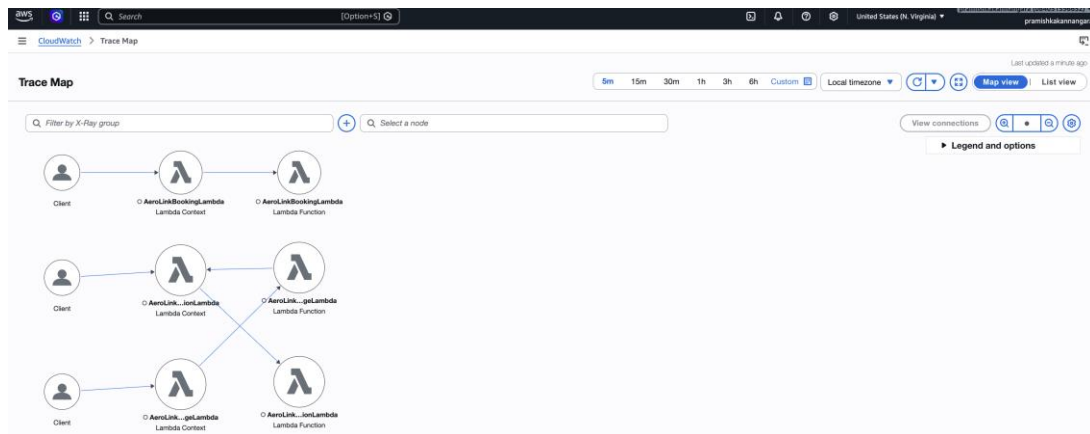


Figure 72: AWS X-Ray service map showing distributed tracing across AeroLink cloud services.

Logging configuration [Info](#)

Log destination

CloudWatch log group

[/aws/lambda/AeroLinkBaggageLambda](#) [↗](#)

Log content

Log format

Text

Additional monitoring tools [Info](#)

Logs and metrics (default)

Enabled

CloudWatch Application Signals and AWS X-Ray

Application Signals

Not enabled

Lambda service traces

Enabled

Figure 73: AWS Lambda Active Tracing enabled for distributed tracing using AWS X-Ray.

8.5 Fault Diagnosis Example

It can be noted from the above-discussed points that the KPIs discussed above are important for capacity planning and enhancing performance. Besides, fault detection and service monitoring can be well understood using the example below about the status information of baggage updates in

the case of AeroLink. As soon as there is an update of the status information about the baggage in Baggage Services, the event `BAGGAGE_UPDATED` is triggered on Amazon EventBridge. It follows that the notifications service gets triggered after the event occurs and stores all the notifications on DynamoDB. CloudWatch monitors all these processes discussed above and aids in fault detection in events. It can be concluded that it has been made evident to us that event-driven architecture allows transparency in the whole process.

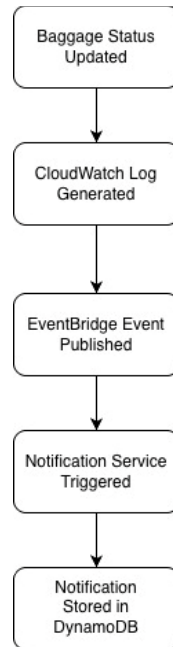


Figure 74: Fault diagnosis workflow using CloudWatch logs and traces.

The screenshot shows the AWS DynamoDB console interface. At the top, there's a search bar and navigation tabs. The main section is titled 'Scan or query items' with a 'Scan' button selected. Below this, there are dropdowns for 'Select a table or index' (set to 'Table - AeroLinkNotifications') and 'Select attribute projection' (set to 'All attributes'). There are also 'Run' and 'Reset' buttons. A green status bar indicates 'Completed - Items returned: 2 - Items scanned: 2 - Efficiency: 100% - RCUs consumed: 2'. Below this, a table titled 'Table: AeroLinkNotifications - Items returned (2)' shows two rows of data. The first row has an ID of '1779790966717', a createdAt timestamp of '2026-05-2...', a message 'Baggage BAG-AL100-001 status changed to Arrived at Destination', and a type of 'BAGGAGE_UPDATED'. The second row has an ID of '1779789302642', a createdAt timestamp of '2026-05-2...', a message 'Notification Lambda working', and a type of 'TEST_NOTIFICATION'.

id (String)	createdAt	message	type
1779790966717	2026-05-2...	Baggage BAG-AL100-001 status changed to Arrived at Destination	BAGGAGE_UPDATED
1779789302642	2026-05-2...	Notification Lambda working	TEST_NOTIFICATION

Figure 75: Notification records generated after baggage status updates.

9 Testing Strategy

9.1 Unit Testing

Testing was done for individual units in the Jest framework. The first goal of unit testing is to make sure that the individual microservices function properly without any problem when they are not yet combined with other microservices. The automated tests were designed to check different services like the Authentication Service, Flight Service, Booking Service, Baggage Service, Notification Service, Payment Service, Check-in Service, and Airport Service. In other words, automated tests were intended to verify the business logic, data validation, authentication, status, seat calculation, payment verification, boarding pass generation, and events creation at the airport.

The primary objective of unit testing was to examine the performance of the microservices with the help of valid and invalid data input. All the automated tests were successful. This demonstrates the correctness and reliability of the core functionality implemented within each service.

4.Summary of Testing Activities and Outcomes

Service	Test case	Expected result	Result
Authentication	User Login	JWT token generated	Pass
Authentication	Invalid Login	Authentication denied	Pass
Booking	Seat Availability Update	Available seats reduced	Pass
Baggage	Status Update	Baggage status updated	Pass
Notification	Notification Creation	Notification generated	Pass
Flight	Flight Seat Calculation	Seat count updated	Pass
Payment	Payment Simulation	Payment approved	Pass
Payment	Invalid Card Validation	Payment rejected	Pass
Check-In	Boarding Pass Generation	Boarding pass generated	Pass
Airport Integration	Airport Event Creation	Event recorded	Pass

```
pramishkakannagaram2@192 testing-evidence % npm test
> test
> jest
PASS testing-evidence/test/unit/checkin.test.js
PASS testing-evidence/test/unit/booking.test.js
PASS testing-evidence/test/unit/airport.test.js
PASS testing-evidence/test/unit/flight.test.js
PASS testing-evidence/test/unit/notification.test.js
PASS testing-evidence/test/unit/auth.test.js
PASS testing-evidence/test/unit/payment.test.js
PASS testing-evidence/test/unit/baggage.test.js

Test Suites: 8 passed, 8 total
Tests: 10 passed, 10 total
Snapshots: 0 total
Time: 1.231 s
Ran all test suites.
```

Figure 76: Automated Jest unit testing results for AeroLink microservices

Eight Jest tests were conducted altogether to yield ten successful test results without experiencing any failure. The test results prove that all microservices operate as desired prior to their incorporation into the cloud architecture.

9.2 Integration Testing

In the case of integration testing, the objective was to ascertain whether the interactions had been achieved successfully between the AeroLink microservices and the AWS Cloud services. For verifying whether the request from the microservice could be authorized by the use of JWT token, it was necessary for the establishment of connectivity between the Authentication Service and API Gateway. For service interactions between two microservices, and in order to verify the availability of seats for booking, the Booking Service and Flight Service were integrated. The events of Baggage Service were sent to the Amazon EventBridge, from where they were transferred to the Notification Service. The authorization process was followed, and thereafter, payment details were stored in a safe way in the DynamoDB database. The findings showed the successful service interaction and secure data exchange through the distributed cloud system.

5: Integration Testing Summary

Integration scenario	Expected result	Result
API Gateway → Auth Service	JWT issued	Pass
Booking Service → Flight Service	Seat validation successful	Pass
Payment Service → DynamoDB	Transaction stored	Pass
Baggage Service → EventBridge	Event published	Pass
EventBridge → Notification Service	Notification generated	Pass

Table: AeroLinkPayments - Items returned (5)

Scan started on June 03, 2026, 15:34:00

☐	id (String)	amount	bookingId	createdAt	last4	provider	status	transactionRef
☐	1780461708713	100	TEST-1780461707836	2026-06-0...	1111	Simulated ...	PAYMENT_APPROVED	TXN-1780461708713
☐	1780033768502	1500	TEMP-1780033767374	2026-05-2...	6209	Simulated ...	PAYMENT_APPROVED	TXN-1780033768502
☐	1780211907053	500	TEST-BOOKING-001	2026-05-3...	1111	Simulated ...	PAYMENT_APPROVED	TXN-1780211907053
☐	1780032788503	500	TEST-BOOKING-001	2026-05-2...	1111	Simulated ...	PAYMENT_APPROVED	TXN-1780032788503
☐	1780033838074	500	TEMP-1780033837315	2026-05-2...	6209	Simulated ...	PAYMENT_APPROVED	TXN-1780033838074

Figure 77: Payment transaction records successfully stored in Amazon DynamoDB following integration testing of the AeroLink Payment Service.

9.3 API Testing

For the testing of API, the following tests have been conducted based on the documentations of Swagger and API Gateway. As part of the testing process, valid payloads were sent to test the routing of the requests and responses and their corresponding status codes. It has been observed from the outputs from the testing process that there has been success in routing the request to the AWS Lambda function using API Gateway. Successful responses have been received from various APIs such as Authentication, Flight, Booking, Baggage, Notification, Check-in, and Payment.

POST

/baggage

Create baggage record

Parameters

Cancel

No parameters

Request body required

application/json

Edit Value

```
{  "bookingId": "1",  "passengerName": "Pranishka",  "tagNumber": "BAG-AL100-001"}}
```

ExecuteClear

Responses

Curl

```
curl -X 'POST' \  https://yzzf3ys00.execute-api.us-east-1.amazonaws.com/prod/baggage' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "bookingId": "1",  "passengerName": "Pranishka",  "tagNumber": "BAG-AL100-001"}'
```

Request URL

https://yzzf3ys00.execute-api.us-east-1.amazonaws.com/prod/baggage

Server response

Code	Details
201	Response body <pre>{"message": "Baggage record created successfully", "baggage": [{"id": "1780480405107", "bookingId": "1", "passengerName": "Pranishka", "tagNumber": "BAG-AL100-001", "userEmail": "unknown", "status": "Checked-In", "lastUpdated": "2026-06-03T09:53:25.107Z"}]}</pre> Download Response headers <pre>content-length: 239 content-type: text/plain; charset=utf-8</pre>

Responses

Code	Description	Links
201	Baggage created successfully	No links

Figure 78:Baggage Service API Testing

POST

/checkins

Passenger check-in and boarding pass generation

Parameters

No parameters

Request body required

application/json

Edit Value

```
{
  "bookingId": "TEST-BOOKING-001",
  "flightId": "1",
  "passengerName": "Pranishka"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'https://yzfq3ys00.execute-api.us-east-1.amazonaws.com/prod/checkins' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "bookingId": "TEST-BOOKING-001",
    "flightId": "1",
    "passengerName": "Pranishka"
  }'
```

Request URL

https://yzfq3ys00.execute-api.us-east-1.amazonaws.com/prod/checkins

Server response

Code	Details
201	Response body <pre>{ "message": "Passenger check-in completed successfully", "checkin": { "id": "1790480483281", "bookingId": "TEST-BOOKING-001", "flightId": "1", "passengerName": "Pranishka", "userEmail": "unknown", "checkInStatus": "CHECKED_IN", "boardingPassNumber": "BP-1780480483281", "seatNumber": "A29", "gate": "G12", "createdAt": "2026-06-03T09:54:43.281Z" } }</pre> Response headers <pre>content-length: 324 content-type: text/plain; charset=utf-8</pre>

Responses

Code	Description	Links
201	Check-in completed successfully	No links

Figure 79: Check-In Service API Testing

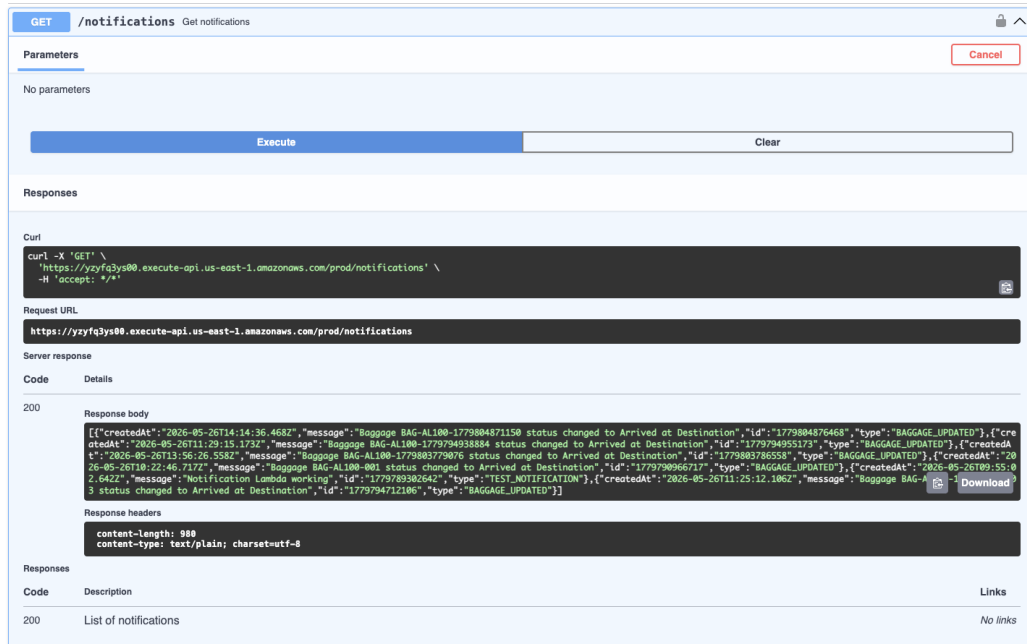


Figure 80: Notification Service API Testing

6: API Testing Results for AeroLink Microservices

API endpoint	Method	Status code	Result
/auth/login	POST	200	Pass
/auth/register	POST	201	Pass
/flights	GET	200	Pass
/bookings	POST	201	Pass
/check ins	POST	201	Pass
/baggage	GET	200	Pass
/payments/simulate	POST	200	Pass
/notifications	GET	200	Pass

```

pramishkakannagaram2@192 aerolink-platform % cd ~/aerolink-platform/testing-evidence
npm install axios
node api-tests.js

added 27 packages, and audited 35 packages in 2s

7 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PASS - Auth Login returns JWT
PASS - GET Flights returns list
PASS - Bookings without JWT is rejected
PASS - GET Bookings with JWT succeeds
PASS - Payment simulation succeeds
PASS - GET Baggage with JWT succeeds
PASS - GET Notifications with JWT succeeds

```

Figure 81: Automated API test execution results using Axios for AeroLink microservices.

9.4 Security Testing

The process of security testing was performed to confirm that authentication, authorization, and access control functionalities have been performed successfully. To gain access to the secure APIs, the JWT tokens from the Authentication service were utilized. Through this process, those requests having the appropriate JWT token gained access, whereas those requests without any token or those containing an incorrect token did not get access. It has also been tested whether Role-Based Access Control works efficiently in terms of ensuring that only authorized individuals use the admin feature.

```

pramishkakannagaram2@192 testing-evidence % curl -X POST \
https://zyzf3ys00.execute-api.us-east-1.amazonaws.com/prod/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "admin@aerolink.com",
  "password": "Admin123"
}'
{"message": "Login successful", "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWVpbGUiOiJhZG1pb1IsImhhdCI6MTc4MDQ4MTU0MiwiZXhwIjoxNzg1NTQyfq0.ogu5uJqaqHrBuFo-waRoT-fG7SV5wC8lWfgQe2spaTQ"}

```

Figure 82: JWT token generation (/auth/login)

```

pramishkakannagaram2@192 testing-evidence % curl -i \
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWVpbGUiOiJhZG1pb1IsImhhdCI6MTc4MDQ4MTU0MiwiZXhwIjoxNzg1NTQyfq0.ogu5uJqaqHrBuFo-waRoT-fG7SV5wC8lWfgQe2spaTQ" \
https://zyzf3ys00.execute-api.us-east-1.amazonaws.com/prod/bookings
HTTP/2 200
date: Wed, 03 Jun 2026 10:13:07 GMT
content-type: text/plain; charset=utf-8
content-length: 11299
apiw-requestid: eYSn813JIAMEPUw=

```

Figure 83: Protected endpoint access with valid JWT (HTTP 200)

```

pramishkakannagaram2@192 testing-evidence % curl -H "Authorization: Bearer INVALID_TOKEN" \
https://zyzf3ys00.execute-api.us-east-1.amazonaws.com/prod/bookings
{"message": "Access denied. Valid JWT token required."}

```

Figure 84: Unauthorized access attempt (Access denied. Valid JWT token required.)

9.5 End-to-End Testing

The tests done in end-to-end involved checking whether all the business processes were running smoothly across all microservices. In the process of booking, there were processes of authentication of the user, flight selection, authorizations for payments, confirmation of the booking, and also information about seat availability. Another business process involved the baggage process, which consisted of baggage registration, status information, and also event creation using EventBridge.

10 Challenges Encountered

However, there were certain issues encountered when designing and implementing AeroLink.

One of the difficulties that we encountered was related to the implementation of the multiple microservices on the basis of AWS. The correct launch of the Amazon ECS Fargate services, as well as the creation of the task definitions, and the correct interaction of all the services between each other were the difficult tasks to complete.

The second difficulty that we have met in the course of our work on implementing the AeroLink system is related to the implementation of the AWS API Gateway, including setting up routing rules for sending requests to AWS Lambda Functions and other back-end components.

There was also an issue in connection with API gateway configuration and handling requests. It was required to configure some REST APIs for proper handling and distribution of incoming requests to AWS Lambda functions and backend services.

Lastly, the use of JWT-based authentication and authorization mechanisms was not an easy job. Generating JWTs correctly and ensuring role-based access control were important to make sure that only proper API calls can be made by authorized users. While preventing any unauthorized access had to be done cautiously.

The deployment of Amazon EventBridge also added an element of difficulty. Configuring the publication of events, setting up event routing rules, and managing notifications became necessary to facilitate effective asynchronous communication among different services. Debugging and monitoring of events' handling process proved to be very crucial.

Database design posed another set of challenges. The design of DynamoDB tables had to be considered such that they could support good scalability and performance of queries. Choosing appropriate partition keys and designing schemas that fit distributed systems proved to be very challenging.

11 Future Improvements

However, despite the effectiveness of AeroLink being confirmed by its ability to perform the set of tasks through a cloud-based distributed computing system, there are still some improvements that could be made in the future concerning this solution. This will include the use of Amazon EKS (Amazon Elastic Kubernetes Service), providing the management and orchestration tools needed to manage this software through the Kubernetes container services for microservices (Amazon.com, 2026). Moreover, the integration of artificial intelligence and machine learning technologies will make it possible to predict the number of passengers and flights required (Geske, Herold, & Kummer, 2024).

An additional improvement to be made in the future includes the implementation of enhanced analytics and reporting functionalities. This will be facilitated by incorporating services such as AWS QuickSight (Rahul Awati and Carty, 2021). The inclusion of such services would help make the process of generating reports much easier through the provision of dashboard analytics and business intelligence for the airline's management team (Amazon Web Services, Inc., 2026).

Finally, the AeroLink integration would also allow the adoption of either a multi-tenant or multi-cluster configuration that would lead to increased reliability and continuity of operations. This would entail the deployment of the services among several CSPs, making the system independent of one particular CSP and increasing its availability in case of any possible disruption (Karl, 2024). In addition to this, other forms of automation based on events such as delays, changes, baggage exceptions, and passenger notifications can be introduced (Redhat.com, 2024).

12 Conclusion

The AeroLink project was successfully developed and deployed in the form of distributed airline management software based on Amazon Web Services (Amazon Web Services, Inc., 2026). The main improvement made by the transition to distributed architecture includes the ability to scale, deploy independently, maintain the software, and improve its availability (GeeksforGeeks, 2024).

The use of event-driven architecture is highly effective due to such services as RESTful API, AWS Lambda, Amazon API Gateway, ECS Fargate, DynamoDB, and Amazon EventBridge (Amazon Web Services, Inc., 2026). Secure communication between different software elements is provided with the help of HTTPS, JWT authentication, and RBAC authorization services.

Testing through unit testing, integration testing, API testing, security testing, performance testing, and end-to-end testing ensured the reliability and functionality of the system (Verma, 2024). Implementation of monitoring and observability capabilities was achieved using AWS CloudWatch to enable visibility of the system's behavior and performance (nOps, 2024).

In conclusion, AeroLink shows that cloud-native technology is capable of providing the means of developing secure, scalable, fault-tolerant, and highly available distributed applications (Kroeze, 2024). This project serves as a good starting point for making improvements in the future.

Appendix - System Access Links

Recourses	URL
AeroLink webpage	<u>https://d1golwq2x99wr4.cloudfront.net/</u>
Swagger/OpenAPI Documentation	<u>https://d1golwq2x99wr4.cloudfront.net/swagger-ui/index.html</u>
GitHub Repository	<u>https://github.com/pramishka22/AeroLink.git</u>

References

GeeksforGeeks (2024). *AWS Amazon EventBridge*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/devops/aws-eventbridge/> [Accessed 19 May 2026].

Amazon.com. (2019). *Architect for AWS Fargate for Amazon ECS - Amazon Elastic Container Service*. [online] Available at: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html [Accessed 20 May 2026].

Amazon.com. (2026). *What is Amazon Aurora? - Amazon Aurora*. [online] Available at: https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_AuroraOverview.html [Accessed 20 May 2026].

Amazon Web Services, Inc. (2026). *Web Application Firewall, Web API Protection - AWS WAF - AWS*. [online] Available at: <https://aws.amazon.com/waf/> [Accessed 21 May 2026].

K21 Academy. (2023). *AWS WAF: Setup, Rules & Pricing Guide 2026 | K21 Academy*. [online] Available at: <https://k21academy.com/aws-cloud/what-is-aws-waf/> [Accessed 21 May 2026].

Amazon Web Services, Inc. (2026). *Cloud Storage Services on AWS*. [online] Available at: https://aws.amazon.com/products/storage/?trk=f45cade8-86c2-403d-b03b-e9392f982647&sc_channel=ps&trk=170f57ba-3277-4e3e-aa17-e32fcfd1b2b9&sc_channel=ps&ef_id=Cj0KCQjwof_QBhCgARIsADaMzOdnMqQHt9EHIO-k-YEzOmMcHb3pPCdOFRMJVU10Pq8eD9ZmDpWyHMsaAqcHEALw_wcB:G:s&s_kwid=AL!4422!3!795924543491!e!!g!!aws%20web%20storage!23523527220!198422377331&gad_campaignid=23523527220&gbraid=0AAAAADjHtp9pXK17bHtpwezBXSkp6JSbD&gclid=Cj0KCQjwof_QBhCgARIsADaMzOdnMqQHt9EHIO-k-YEzOmMcHb3pPCdOFRMJVU10Pq8eD9ZmDpWyHMsaAqcHEALw_wcB [Accessed 21 May 2026].

GeeksforGeeks (2024). *Monolithic Architecture*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/monolithic-architecture-system-design/> [Accessed 21 May 2026].

Amazon Web Services, Inc. (2026). *Free Container Services - AWS*. [online] Available at: https://aws.amazon.com/free/containers/?trk=3eec97b6-f16a-46af-9de8-d97305dc8014&sc_channel=ps&trk=6a10a6f9-79d6-43ec-9e76-3cfd9ed0e028&sc_channel=ps&ef_id=Cj0KCQjwof_QBhCgARIsADaMzOdXMLMyHJfa2XZ ZQKn1TRgOVSDJxaJBhbt7mOntNFroFtIqCPksQBsaAuFjEALw_wcB:G:s&s_kwid=AL!4422!3!795924654890!p!!g!!container%20orchestration%20tools!23533254916!192628940133&gad_campaignid=23533254916&gbraid=0AAAAADjHtp8d4dYvi7UpC7FS-

WNQGRban&gclid=Cj0KCQjwof_QBhCgARIsADaMzOdXMLMyHJfa2XZZQKn1TRgOVSDJxaJBhbt7mOntNFroFtIqCPksQBsaAuFjEALw_wcB [Accessed 22 May 2026].

Verma, A. (2024). *Unit Testing of React Apps using JEST : Tutorial | BrowserStack*. [online] BrowserStack. Available at: <https://www.browserstack.com/guide/unit-testing-of-react-apps-using-jest> [Accessed 22 May 2026].

nOps. (2024). *nOps*. [online] Available at: <https://www.nops.io/glossary/what-is-aws-cloudwatch/> [Accessed 23 May 2026].

Kroeze, N. (2024). *What is Cloud Native? A Comprehensive Guide*. [online] Intercept.cloud. Available at: <https://intercept.cloud/en-gb/blogs/what-is-cloud-native> [Accessed 23 May 2026].

Amazon.com. (2026). *What is Amazon EKS? - Amazon EKS*. [online] Available at: <https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html> [Accessed 23 May 2026].

Geske, A.M., Herold, D.M. and Kummer, S. (2024). Artificial intelligence as a driver of efficiency in air passenger transport: A systematic literature review and future research avenues. *Journal of the Air Transport Research Society*, [online] 3, p.100030. doi:10.1016/j.jatrs.2024.100030.

Rahul Awati and Carty, D. (2021). *Amazon QuickSight*. [online] Search AWS. Available at: <https://www.techtarget.com/searchaws/definition/Amazon-QuickSight> [Accessed 25 May 2026].

Amazon Web Services, Inc. (2026). *Analytics on AWS*. [online] Available at: https://aws.amazon.com/big-data/datalakes-and-analytics/?trk=80f57291-7613-4e10-be58-2e9f8325344d&sc_channel=ps&trk=6a2236c1-f996-415b-917a-ddc149c871f1&sc_channel=ps&ef_id=Cj0KCQjwof_QBhCgARIsADaMzOdS0xRSjZ9ubuFcLkMYhd2lembMzGDAywq0SxF5jQAhpbkIRCRv-8kaAILnEALw_wcB:G:s&s_kwid=AL!4422!3!795924543593!p!!g!!data%20strategy%20tools!23528572238!195473573929&gad_campaignid=23528572238&gbraid=0AAAAADjHtp-rCKG3FcEOahifYTAyXY4jo&gclid=Cj0KCQjwof_QBhCgARIsADaMzOdS0xRSjZ9ubuFcLkMYhd2lembMzGDAywq0SxF5jQAhpbkIRCRv-8kaAILnEALw_wcB [Accessed 25 May 2026].

Karl, T. (2024). *What Is Multi-Cloud Architecture? Advantages, Challenges, and Best Practices*. [online] New Horizons. Available at: <https://www.newhorizons.com/resources/blog/multi-cloud-architecture> [Accessed 26 May 2026].

Redhat.com. (2024). *What is event-driven automation?* [online] Available at: <https://www.redhat.com/en/topics/automation/what-is-event-driven-automation> [Accessed 28 May 2026].